

PYTHON FUNDAMENTALS

- python is one of the most popular programming languages.
- It's simple to use, Packed with features and supported by a wide range of libraries and frameworks.
- It's clear syntax and makes it beginner friendly.
- A high-level language, used in data science, Automation, AI, web development and more.
- Known for its readability which means code is easier to write, understand and maintain.
- Backed by strong library support, we don't have to build everything from scratch.

Why learn python?

- Requires fewer lines of code compared to other programming languages.
- provides libraries and frameworks such as Django and Flask for web development and tools like pandas, Tensorflow and scikit-learn for artificial intelligence, ML and data analysis.
- cross-platform, works on windows, Mac and Linux without major changes.
- used by top tech companies like Google, Netflix and NASA.

Introduction :

- Python is a high-level programming language known for its simple and readable syntax. It has the following features:
- Allows writing clean code with fewer lines.
- supports multiple programming paradigms including object oriented, functional and procedural programming.
- widely used in web development, automation, data analysis and AI and many other fields.
- Dynamically typed and has automatic garbage collection.

Ex: `print("Hello, World!");`

`print()` — a built-in python function that instructs the computer to display text on the screen.

"Hello, World!" — a string enclosed with in quotes.

Python uses indentation to define code blocks instead of braces in C, C++ and Java.

`#` is used to write comment on python.

Comments :-

- Comments in python are used to add explanatory notes to code and are ignored during execution.
- It enhance the readability of the code.
- It can be used to identify functionality.
- It can be used to prevent executing any specific part of your code, while making changes.

single-line comments

• single line comments starts with hashtag. #

Ex:

```
# sample comment
```

```
name = "Harika"
```

```
print(name)
```

o/p: Harika

Multi-line comments

- python does not have a dedicated syntax for multi-line comments.
- However they are commonly written using multiple #'s or unassigned string literals.

Ex: # python program

```
# multiline comments
```

```
print("Harika")
```

o/p: Harika

String Literals:

- Literals in python are fixed values written directly in the code that represent constant data.
- They provide a way to store numbers, text or other info that does not change during execution.
- python supports different types of literals, such as numeric literals, string literals, boolean literals and special values like none.

Ex:

- 10, 3.14, 5+2j are numeric literals.
(int) (float) (complex)

- "Hello" & "python" are string literals.

- True and False are boolean literals.
(1) (+) (0) (-)

There are 4 types of string literals.

1. single-quoted strings — enclosed in single quotes ('Hello')
2. Double-quoted strings — enclosed in double quotes ("Python")
3. Triple-quoted strings — enclosed in triple quotes ("''Hello world'')
4. Raw string — prefix with r to ignore escape sequences.
(\n, \t, etc.)

ex: r"C:\Users\Python"

'None' is used to define a null variable. It is a special literal.

```
res = None
print(res)
```

Types of Literals	Description	Example	mutable/immutable
1. Integer Literals	whole numbers (without decimals)	a = 77	Immutable
2. Float Literals	Numbers with a decimal point	b = 3.14	Immutable
3. complex Literals	Numbers with real and imaginary part	c = 7+5j	Immutable
4. String Literals	Stores text. Can use single, double or triple quotes	greeting = "Hello"	Immutable
5. Boolean Literals	Represents true(1) or false(0)	a = (1 == true) → true b = (1 == false) → false	Immutable
6. Boolean as Number	Boolean values can act as number(1 or 0)	c = True + 3 → 4 d = false + 7 → 7	Immutable
7. List Literal	Stores multiple values; can be changed	Rank = ["First", "Second", "Third"]	Mutable
8. Tuple Literal	Like a list but cannot be changed.	colors = ("Red", "Blue", "Green")	Immutable
9. Dictionary Literal	Stores key-value pairs	class = {"Jai": 10, "Sai": 12}	Mutable
10. Set Literal	Stores unique values, unordered.	unique_num = {1, 2, 3}	Mutable
11. Special Literal	Represents "nothing" or "empty".	wake_remain = None	Immutable

Application of Python:

- web development - frameworks like Django, Flask
- ML and AI - tensorflow, pyTorch, scikit-learn.
- Game development - Libraries like py.game
- Desktop Applications - GUI frameworks like Tkinter, Pyat.
- Internet of Things - micropython, Raspberry Pi.
- Devops and cloud - Automation script and APIs.
- CyberSecurity - Penetration testing and ethical hacking tools.
- Data Science and Analysis - Libraries like pandas, numpy, Matplotlib.

Advantages of python:

1. Easy to learn.
2. object oriented and procedural programming language.
3. Dynamically typed language. [Don't need to declare data types explicitly]
4. open source and large active community base.
5. Extensive support libraries.
6. presence of third-party modules.
7. Versatile.
8. cross platform.

DisAdvantages of Python:

1. Slower Execution Speed.
2. Higher memory usage.
3. Runtime errors.
4. Not ideal for mobile development.
5. Memory consumption.
6. Global Interpreter lock.
7. Not ideal for some domains.

Real-world use case:

1. YouTube - for videostreaming and backend services.
2. Instagram - for scaling and handling millions of users.
3. Spotify - for backend services and ml to personalize music recommendations.
4. Netflix
5. Google
6. Uber
7. Pinterest

Input and Output in python:

`print()` function is used for output in various formats.
`input()` function enables interaction with users.

Taking Input using input()

python's `input()` function is used to take user input.
By default, it returns the user input in form of a string.

```
-> name = input("Enter your name: ")
```

```
print("Hello", name, "! Welcome!")
```

```
o/p: Enter your name: Harika
```

```
Hello, Harika! Welcome!
```

Printing Output using print()

`print()` function allows us to display text, variables and expression on the console.

```
o/p: print("Hello, world!");
```

```
o/p: Hello, world
```

Printing Variables

• we can use the `print()` function to print single and multiple variables.

• we can print multiple variables by separating them with commas.

```
Ex: s = "Harika"
```

```
print(s)
```

```
s = "Sravani"
```

```
age = 20
```

```
city = "New York"
```

```
Print(s, age, city)
```

```
o/p: Harika
```

```
Sravani 20 NewYork
```

Taking multiple inputs from user in python using

input() and split()

• one of the simplest ways to take multiple inputs from a user in python is by using the `input()` function along with the `split()` method.

• The `split()` method splits a string into a list based on a specific separator (by default, it uses whitespace).

Ex:

```
x, y, z = input("values:").split() → divide the string into  
separate components  
based on whitespaces  
by default.  
print(x)  
print(y)  
print(z)
```

o/p: values: 5 6 7

5

6

7

Taking Different types of user input

• we can change the user input from default string type to any other type (int, float etc) by typecasting

```
i = int(input("How old are you?"))
```

```
f = float(input("Evaluate 7/2: "))
```

```
print(i, f)
```

o/p: How old are you? 20

Evaluate 7/2: 3.5

20 3.5

change the type of input in python:

1. Taking Integer inputs

we can typecast the user input to integer type by wrapping the input statement by int():

```
Ex: n = int(input("How many roses?"))
```

```
print(n, type(n))
```

o/p: How many roses?: 88

88 <class 'int'>

2. Taking float or decimal inputs

• we can typecast the user input to float or decimal type by wrapping the input statement by float():

```
Ex: price = float(input("Price of each rose?"))
```

```
print(price, type(price))
```

o/p: price of each rose?: 69.420

69.420 <class 'float'>

TYPECASTING :

• TYPECASTING is the method to convert the python variable datatype into a certain data type in order to perform required operations by users.

• There can be two types of TYPECASTING in java.

1. Implicit TYPECONVERSION.

2. Explicit TYPECONVERSION

1. Implicit TYPE CONVERSION

• Implicit type conversion is the process of converting a smaller data type into a larger compatible data type during an operation.

• It is an automatic conversion.

Ex:

```
a = 7
print(type(a))
b = 3.0
print(type(b))
c = a + b
```

o/p: `<class 'int'>`

`<class 'float'>`

10.0

`<class 'float'>`

```
print(c)
```

```
print(type(c))
```

• a is an int, b is a float. When adding them, python automatically converts a from int to float.

• This automatic conversion is called implicit type casting.

2. Explicit TYPE CONVERSION

• The programmer converts one data type into another using built-in functions such as `int()`, `float()`, `str()` etc.

• Commonly used type casting function in python are:

- `int()` - `int()` function takes float or string as an argument and returns int type object.

- `float()` - `float()` function takes int or string as an argument and returns float type object.

- `str()` - `str()` function takes float or int as an argument and returns string type object.

convert int to float - converting int to float using `float()`

Ex:

```
a = 5
n = float(a)
```

```
print(n)
```

```
print(type(n))
```

5.0

`<class 'float'>`

Convert float to int

- Converting float to int datatype in python with int() function

```
Ex: a = 5.9
n = int(a)
print(n)
print(type(n))
```

O/P: 5
<class 'int'>

Convert int to String

- Converting int to String datatype in python with str() function

```
Ex: a = 5
n = str(a)
print(n)
print(type(n))
```

O/P: 5
<class 'str'>

Convert String to float

- Converting string data type into float data type with float()

```
Ex: a = "5.9"
n = float(a)
print(n)
print(type(n))
```

O/P: 5.9
<class 'float'>

Convert String to int

- Converting string datatype to int with int() function.
- If given string is not number, then it will throw an error.

```
Ex: a = "5"
b = "t"
n = int(a)
print(n)
print(type(n))
print(int(b))
print(type(b))
```

O/P: 5
<class 'int'>

} → ValueError: invalid literal for int() with base 10: 't'

Adding String and Integer without Conversion

- If we try to directly add an integer and a string, python will throw an error becz they are different data types.

```
Ex: a = 5
b = 't'
n = a + b
print(n)
print(type(n))
```

O/P: TypeError: unsupported operand type(s) for +: 'int' and 'str'

Here we should first convert the string into an integer then add:

Ex: $a = 5$
 $b = '7'$

#explicit conversion of string to int

$n = a + \text{int}(b)$

$\text{print}(n)$

$\text{print}(\text{type}(n))$

O/P:

12
<class 'int'>

Variables:

- Variables are used to store data that can be referenced and manipulated during program execution.
- A variable is essentially a name that is assigned to a value.
- Unlike Java and many other languages, Python variables do not require explicit declaration of type.
- Type of the variable is inferred based on the value assigned.

Ex: $x = 5$

$\text{name} = \text{"Alex"}$

$\text{print}(x)$

$\text{print}(\text{name})$

O/P:

5
Alex

Rules for Naming variables:

- Names can contain letters, digits and underscore.
- The first character cannot be a digit.
- Names are case-sensitive, so `myVar` and `myvar` are treated differently.
- Keywords such as `if`, `else` and `for` cannot be used as variable names.

Ex: $\text{age} = 21$
 $\text{_colour} = \text{"red"}$
 $\text{total_score} = 90$

} These are valid.

Ex: $\text{name} = \text{"Error"}$
 $\text{\#starts with digit}$
 $\text{class} = 10$
 $\text{\#class is a reserved key word.}$
 $\text{user_name} = \text{"Joe"}$
 $\text{\#contains a hyphen.}$

} These are Invalid

Assigning values to Variable:

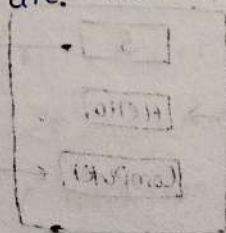
- Basic Assignment - variables are assigned values using

the `=` operator.

Ex: $x = 5$

$y = 3.14$

$z = \text{"Hi"}$



2. Dynamic Typing - python is dynamically typed, so the same variable can store different data type during execution.

Ex ÷ `x = 10`

`x = "Now a string"`

3. Assigning Same Value - same value can be assigned to multiple variables in a single line.

Ex ÷ `a = b = c = 100`

o/p ÷ 100 100 100

`print(a, b, c)`

4. Assigning Different values - Multiple variables can also be assigned different values in a single line.

Ex ÷ `x, y, z = 1, 2.5, "Python"`

o/p ÷ 1 2.5 python

`print(x, y, z)`

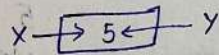
Concept of object Reference:

• let us assign a variable `x` to value 5.

`x = 5`

`y = x`

• Now let's assign another variable `y` to the variable `x`.



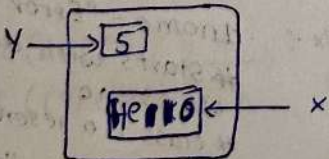
`y = x`

→ This stmt creates `y` and references the same object as `x`, not `x` itself. This is called a shared reference, where multiple variables reference the same object.

• Now if we write,

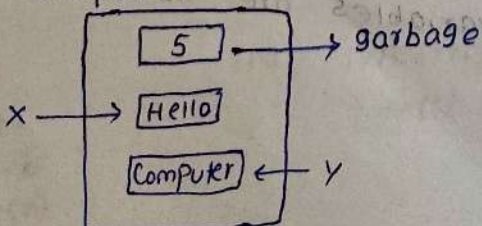
`x = "Hello"`

• Python creates a new object for the value `Hello` and makes `x` reference this new object.



• The variable `y` remains unchanged, still referencing the original object 5. Now if we assign new value to computer `y`:

`y = "computer"`



• python variables store references to objects, not the

actual values themselves, it starts referencing a new object while the old unreferenced object becomes eligible for garbage collection.

variable Reassignment

ex: $x = 1$

$y = x$

$y = y + 1$ O/P: $\div$

print(x) 1

print(y) 2

1. Swapping two variables

using multiple assignments, we can swap the values of two variables without needing a temporary variable.

ex: $a, b = 5, 10$

$a, b = b, a$ O/P: $\div$

print(a, b) 10 5

2. Counting characters in a String

ex: $word = "python"$

$length = len(word)$

print("Length of the word:", length)

O/P: Length of the word: 6

• del removes the variable definition from memory.

Operators :

• operators are used to perform operations on values and variables.

• operators: special symbols like +, -, *, / etc.,

• operands: value on which the operator is applied.

TYPE

Arithmetic operators

Relational operators

Logical operators

Bitwise operator

Assignment operator

Ternary operator

Identity operator

operators

+, -, *, /, %

<, <=, >, >=, ==, !=

AND, OR, NOT

&, |, ^, ~, <<, >>

=, +=, -=, *=, /=

x if condition else y

is, is not

Arithmetic operators

Arithmetic operators are used to perform basic mathematical operations like addition, subtraction, multiplication, Division and modulus.

- In python, the division operator returns a floating-point result.
- floor division (//) returns an integer result.

Ex ÷

a = 15

b = 4

print ("Addition:", a + b)

print ("Subtraction:", a - b)

print ("Multiplication:", a * b)

print ("Division:", a / b)

print ("Floor Division:", a // b)

print ("Modulus:", a % b)

print ("Exponentiation:", a ** b)

O/P ÷

Addition: 19

Subtraction: 11

Multiplication: 60

Division: 3.75

Floor Division: 3

Modulus: 3

Exponentiation: 50625

Relational operators

- also called as comparison operators to compare values.
- It either returns true or false according to the condition.

Ex ÷

a = 133

b = 33

print(a > b) False

print(a < b) True

print(a == b) False

print(a != b) True

print(a >= b) False

print(a <= b) True

Logical operators

- These operators perform Logical AND, Logical OR and Logical NOT operations.
- It is used to combine conditional statements.
- The precedence of logical operators in python is as follows:

1. Logical not

2. Logical and

3. Logical or

Ex: a = True
 b = False
 print(a and b) False
 print(a or b) True
 print(not a) False

Bitwise operators

- These operators act on bits and perform by ~~sub~~ bit-by-bit operations.
- These are used to operate on binary numbers.
- Bitwise operators in python are as follows:

1. Bitwise NOT
2. Bitwise Shift
3. Bitwise AND
4. Bitwise XOR
5. Bitwise OR

Ex: a = 10
 b = 4
 print(a & b) 0
 print(a | b) 14
 print(~a) -11
 print(a ^ b) 14
 print(a >> 2) 2
 print(a << 2) 40

Assignment Operators

- These are used to assign values to the variables.
- This operator is used to assign the value of the right side of the expression to the left side operand.

Ex: a = 10
 b = a
 print(b) 10
 b += a
 print(b) 20
 b -= a
 print(b) 10
 b *= a
 print(b) 100
 b /= a
 print(b) 10.0
 b **= a
 print(b) 10000

Identity operators

- "is" and "is not" are the identity operators.
- both are used to check if two values are located on the same part of the memory.
- The two variables that are equal do not imply that they are identical.
- is $\rightarrow$ True if the operands are identical
- is not $\rightarrow$ True if the operands are not identical

Ex: $a = 10$

$b = 20$

$c = a$

print (a is not b) True

print (a is c) True

Membership operators

- "in" and "not in" are the membership operators.
- that are used to test whether a value or variable is in a sequence.

Ex: $x = 24$

$y = 20$

my_list = [10, 20, 30, 40, 50]

if (x not in my_list):

 print ("x is NOT present in given list")

else:

 print ("x is present in given list")

if (y in my_list):

 print ("y is present in given list")

else:

 print ("y is NOT present in given list")

o/p:

x is NOT present in given list

y is present in given list.

Ternary operator

- It is also known as conditional expressions are operators that evaluate something based on a condition being true or false.
- it simply allows testing a condition in a single line replacing the multiline if-else, making the code compact.

syntax: `[on-true] if [expression]
else [on-false]`

ex: `a, b = 10, 20
min = a if 'a' < b else b
print(min)`

o/p: 10

Precedence and Associativity of operators in python:

- In python, operators have different precedence levels, which determine order in which expressions are evaluated.
- If operators have same precedence, associativity decides whether they are evaluated left-to-right or right-to-left.

Operator precedence

- It defines order in which python evaluates different operators in an expression.
- When an expression has multiple operators, python follows precedence rules to decide order of evaluation.

expression:

$$\begin{array}{c} 10 + 20 * 30 \\ \quad \quad \quad \downarrow \\ 10 + 600 \\ \quad \quad \quad \downarrow \\ 610 \end{array}$$

Multiply will happen first as
* has higher precedence

Now addition will happen as + has
lower precedence
we have got the solution for the
equation.

ex: `exp = 10 + 20 * 30` o/p: 610
`print(exp)`

Precedence of logical operators

- Logical operators (and, or, not) also follow precedence.

ex: `name = "Alex"
age = 0`

`if name == "Alex" or name == "John" and age >= 2:`

`print("Hello! Welcome.")`

`else:`

`print("Goodbye!!")`

o/p: Hello! Welcome

- and has higher precedence than or.
- So `name == "John" and age >= 2` is checked first
- Since `name == "Alex"` is already true, whole condition passes.
- To fix this, use parentheses:

`if (name == "Alex" or name == "John") and age >= 2`

Here, parentheses ensure the or part is checked first.

operators Associativity

- If an expression contains two or more operators with same precedence then operator associativity is used.
- It can be either left to right or right to left.

Expression:

`100 / 10 * 10` "/" is evaluated first as it is leftmost

$$\begin{array}{c} 100 / 10 * 10 \\ \downarrow \\ 10 * 10 \\ \downarrow \\ 100 \end{array}$$

Now "*" is evaluated.

got the solution.

/ & * both have the same precedence but Left to Right (LTR) Associativity.

Ex: `print(100 / 10 * 10)` o/p: 100.0

`print(5 - 2 + 3)` 6

`print(5 - (2 + 3))` 9

Ex: `print(2 ** 3 ** 2)` o/p: 512

** is evaluated right to left, so `2 ** (3 ** 2) = 2 ** 9 = 512`

operators precedence and Associativity

- These two are main characteristics of operators that determine evaluation order of subexpressions in the absence of brackets.

Expression: `100 + 200 / 10 - 3 * 10`

$$\begin{array}{c} 100 + 200 / 10 - 3 * 10 \\ \downarrow \end{array}$$

`100 + 20 - 3 * 10`

$$\begin{array}{c} 100 + 20 - 3 * 10 \\ \downarrow \end{array}$$

`100 + 20 - 30`

$$\begin{array}{c} 100 + 20 - 30 \\ \downarrow \end{array}$$

`120 - 30`

$$\begin{array}{c} 120 - 30 \\ \downarrow \end{array}$$

`90`

/ happens first as higher precedence.

* happens second as higher precedence. But lower precedence.

+ happens as third precedence

- happens as fourth precedence

got the solution

NON - associative operators

- In python, most operators have associativity, which means they are evaluated from left to right or right to left when they have same precedence.
- non-associative means they cannot be chained together.

Ex: $a = 5$
 $b = 10$
 $c = 15$
 $a = b = (a < b) + = (b < c)$

o/p: `SyntaxError: invalid syntax.`

$=$ & $+=$ are non-associative, combining them in one exp causes a syntax error.

Keywords ÷

- keywords in python are special reserved words that are part of the language itself.
- They define the rules and structure of python programs which means you cannot use them as names for your variables, functions, classes or any other identifiers.

Getting List of all python keywords:

- we can also get all the keyword names using the below code.

tx: `import keyword`
`print("The list of keywords are:")`
`print(keyword.kwlist)`

o/p: The list of keywords are:

`['False', 'None', 'True', 'and', 'as', 'assert', 'async', 'await', 'break', 'class', 'continue', 'def', 'del', 'elif', 'else', 'except', 'finally', 'for', 'from', 'global', 'if', 'import', 'in', 'is', 'lambda', 'nonlocal', 'not', 'or', 'pass', 'raise', 'return', 'try', 'while', 'with', 'yield']`

ways to identify python keywords are:

- with syntax highlighting
- Look for `SyntaxError`

Keywords as Variable Names :

• If we attempt to use a keyword as a variable, python will raise a `SyntaxError`.

Ex : `for = 10` o/p : `Syntax Error : Invalid Syntax`
`print(for)`

Category	Keywords
value keywords	<code>True, False, None</code>
operator keywords	<code>and, or, not, is, in</code>
Control flow keywords	<code>if, else, elif, for, while, break, continue, pass, try, except, finally, raise, assert</code>
Function and class	<code>def, return, lambda, yield, class</code>
context management	<code>with, as</code>
import and module	<code>import, from</code>
Scope and namespace	<code>global, nonlocal</code>
ASync programming	<code>async, await</code>

Keywords vs Identifiers

Keywords	Identifiers
• Reserved words in python that have a specific meaning. • cannot be used as variable names. • Examples: <code>if, else, for, while</code> • Part of the python syntax. • They cannot be redefined or changed.	• Names given to variables, function, classes etc. • can be used as variable names if not a keyword. • Examples: <code>x, number, sum, result</code> • user-defined meaningful names in the code. • can be defined and redefined by the programmer.

Variables or keyword's

variables	Keywords
• Used to store data. • can be created, modified, and deleted by the progr	• Reserved words with predefined meanings in python. • cannot be modified or used as variables

- Examples: x, age, name
- Hold values that are manipulated in the program.
- Variable names must follow naming rules but are otherwise flexible.

- Examples: if, for, while
- Used to define the structure of python code.
- Fixed by python language and cannot be altered.

Tokens:

- A token is the smallest meaningful unit in a python program.
- Python code is interpreted by breaking it into tokens.
- Python has five types of tokens.

1. Keywords
2. Identifiers
3. Literals
4. Operators
5. Punctuator

- ↓
- These are the symbols that are used in python to organize the structures, statements and expressions.
 - Some of the punctuators are: `[ ]`, `{ }`, `( )`, `@`, `-=`, `+=`,

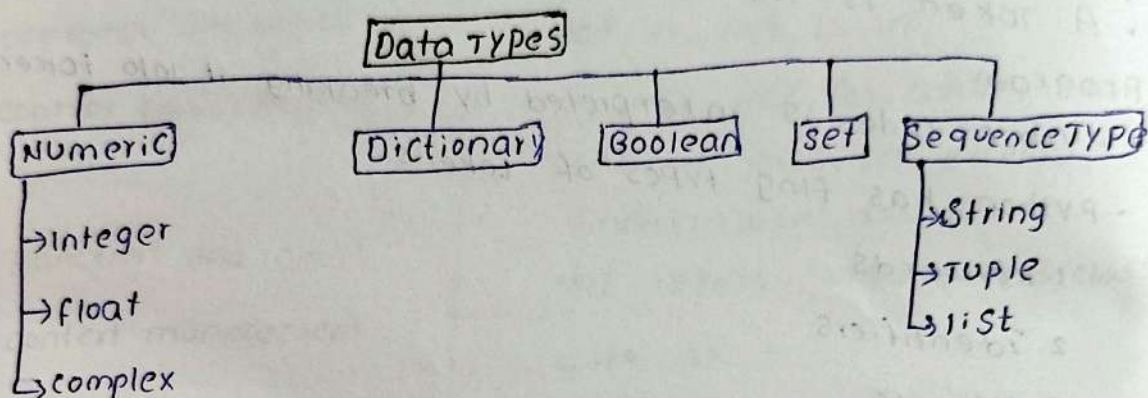
`*=`, `//=`, `**=`, `=`, etc.

Character Set

- It is the collection of valid characters that a programming language understands.
- Python supports a wide range of characters, making it flexible and easy to use.
- Python character set includes:
 1. Alphabets: A-Z, a-z
 2. Digits: 0-9
 3. Special symbols: `+`, `-`, `*`, `/`, `.`, `=`, `@`, `#`, `$`, `&`, `-` etc.
 4. Whitespace characters: space, tab, newline
 5. Unicode characters: Python supports full unicode.
- These characters are used to form keywords, variables, expressions and statements.

Data TYPES :

- Data types in python define the type of value stored in a variable and determine the operations that can be performed on the data.
- Since python treats everything as an object, each value is associated with a specific data type.
- Helps the interpreter understand how to store and process different kinds of data efficiently.



Numeric Data Types

- Numeric data types are used to store numeric values.
- It can be integer, floating number or even a complex number.
- Python supports three main numeric types.

• Integer

- value is represented by int class.

- It contains positive or negative whole numbers. (without fractions or decimals).

• Float

- value is represented by float class.

- It is a real number with a floating-point representation.

- It is specified by a decimal point.

• Complex

- It is represented by complex class.

- It stores numbers with real and imaginary parts.

• For example :

$2 + 3j$
↓ ↓
real imaginary part

Ex:

a=5

b=5.0

c=2+4j

print(type(a)) <class 'int'>

print(type(b)) <class 'float'>

print(type(c)) <class 'complex'>

O/P:

② Dictionary Data Types

- Dictionaries are used to store data in key: value pairs.
- Each key in a dictionary must be unique and value are accessed using their keys with square brackets [] or get() method.
- Dictionaries are case sensitive.

Ex:

d = {1: 'Harika', 2: 'Vennela', 3: 'Sravani', 4: 'Dola'}

print(d[1])

print(d.get(2))

print(d.get(3))

print(d[4])

O/P: Harika

Vennela

Sravani

Dola

③ Boolean Data Types

- Boolean data types represents one of two values: True or False.
- It mainly used in conditions and comparisons and is represented by the "bool" class.

Ex: print(type(True))

print(type(False))

O/P:

<class 'bool'>

<class 'bool'>

Truthy and Falsy values

- Truthy and falsy values are values that evaluate

True or False in a Boolean context.

• Truthy values behave like True.

• Falsy values behave like False when used in condition

Ex:

if 1:

print("1 is truthy")

O/P:

1 is truthy

if not 0:

print("0 is falsy")

0 is falsy

⑤ Set Data Type

• Sets are unordered and mutable collections used to store unique elements.

• Since sets are unordered, elements cannot be accessed using indexing.

• Elements are usually accessed by iterating through the set using a loop.

Ex: $S1 = \{ "a", "a", "b", "c", "b" \}$

print(S1)

$S2 = \{ "Tom", "fox", "Tom" \}$

for i in S2:

print(i)

O/P: $\{ 'b', 'a', 'c' \}$ (or) $\{ 'a', 'b', 'c' \}$ → The order may vary bcs sets are unordered.

Tom

For

⑥ Sequence Data Types

• Sequence is an ordered collection of items, which can be of similar or different data types.

• Elements in a sequence can be accessed using indexing.

• String

• Strings are used to store text data.

• A string is represented using the str class and can be created using single, double or triple quotes.


```

ex: s = 'Welcome to the World' o/p:
print(s)           Welcome to the World
print(type(s))     <class 'str'>
print(s[1])        e
print(s[-1])       d

```

- positive Indexing starts from 0 from the left side.
- Negative Indexing also allows in python it start from the right side using negative numbers.

```

d l r o w - - - - - l e w
-1 -2 -3 -4 -5 - - - - - -17

```

List

- Lists are ordered and mutable collections used to store multiple items in a single variable.
- Elements in a list can be of different data types and are accessed using indexing.

```

ex: a = [1, 2, 3]
print(a)
b = ["Tom", "For", "Tom", 4, 5]
    (0)  (1)  (2)  (3) (4)
print(b[3])
print(b[-3])

```

o/p: [1, 2, 3]

4

Tom

- Negative indexing starts from right and starting with "-1" not 0.

3. Tuple

- Tuples are ordered and immutable collections used to store multiple items in a single variable.
- Once created, tuple elements cannot be modified and are accessed using indexing.

- Tuple with a single element must include a trailing comma, otherwise python treats it as a normal value instead of a tuple.

```
Ex: t1 = (1, )  
print(type(t1))  
t2 = ('Tom', 'for', 'Tom', 1, 2)  
print(t2[3])  
print(t2[-3])
```

```
o/p: <class 'tuple'>  
1  
Tom
```

- Here, $t1 = (1,)$. If we don't write comma, python treats it as an integer, not a tuple.
- So, for a single element tuple, comma is mandatory.
- Here also, Negative indexing starts from the end.

Primitive Data Types vs Non Primitive Data Types :

• Primitive Data Types are the basic building blocks in python.

• They store single values and are immutable, means they can not change after creation.

→ Integer(int) : $x = 10$

→ Float(float) : $y = 10.5$

→ Boolean(bool) : $is_active = True$

→ String(str) : $name = "Alice"$

• These are primitive Data Types.

• Non-primitive data types can store multiple values or complex structures.

• They are generally mutable, means their content can be changed.

→ List(list) : $numbers = [1, 2, 3, 4]$

→ Tuple (tuple) : coordinates = (10.0, 20.0)		
→ Dictionary (dict) : person = {"name": "Alice", "age": 30}		
→ Set (set) : unique_number = {1, 2, 3, 4}		
Differences between primitive and Non-primitive		
Feature	Primitive Datatype	Non-primitive Datatypes
Mutability	Immutable	Mutable
Data representation	Represents a single value	can represent multiple values or structured data.
Example types	int, float, bool, str	list, tuple, dict, set
Memory Allocation	Less memory, as they are simpler	More memory, can store complex data.
methods and operations	Limited operations and methods	Rich set of methods and operations
copying	copying creates new instances.	can create shallow or deep copies.
usecase	simple data, basic calculations	complex data structures, collections.

Conditional Statements:

- conditional statements are used to control the flow of execution in a program based on specific conditions.
- they allow programs to execute different blocks of code depending on whether a condition evaluates True or False.

① If statement

- If statement is used to execute a block of code only when a specified condition evaluates to True.

Syntax:

```
if (condition){
    # block of code
}
```


ex: age = 20

if age >= 18:

print("Eligible to vote")

O/P: Eligible to vote.

② short Hand if

- short Hand if used to write if statements in a single line.
- It is useful when only one statement needs to be executed.

ex: age = 19

if age > 18: print("Eligible to vote")

O/P: Eligible to vote

③ If else statement

- If else statement is used to execute one block of code when the condition is True and another block when the condition is false.

syntax: if (condition){

#block of code

} else {

#block of code

}

Ex: age = 10

if age <= 12:

print("Travel for free")

else:

print("Pay for ticket")

O/P: Travel for free

④ If-elif-else statement

- elif statement is used to check multiple conditions in a program.
- It executes a block of code when its condition evaluates to True after previous conditions evaluates to false.

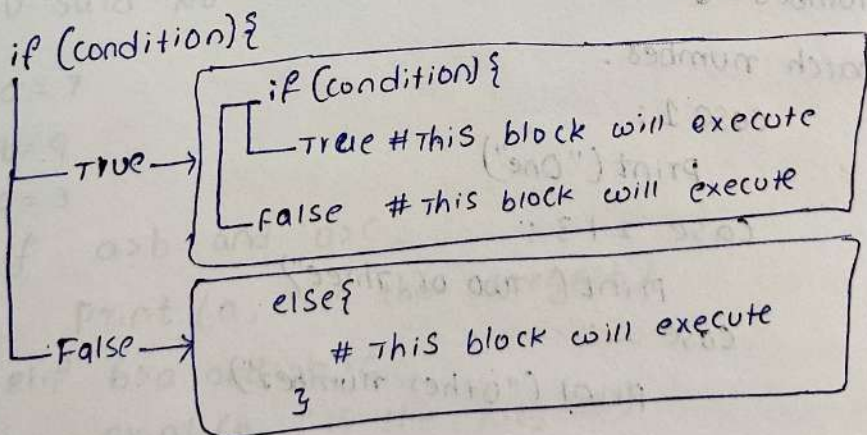
syntax: `if (condition) {`
 `# This block will execute`
 `} else if (condition) {`
 `# This block will execute`
 `} else {`
 `# This block will execute`
 `}`

ex: `age = 25;`
 `if age <= 12:`
 `print ("Child")`
 `elif age <= 19:`
 `print ("Teenager")`
 `elif age <= 35:`
 `print ("Young adult")`
 `else:`
 `print ("Adult")`

o/p: Young adult.

- ① Nested if-else statement
- Nested if-else statement is an if-else statement placed inside another if or else block.
- It is used to check conditions within another condition.

syntax: `if (condition) {`



ex: `age = 70`
 `is_member = True`
 `if age >= 60:`
 `if is_member:`
 `print ("30% Senior discount!")`
 `else:`


```
print("20% Senior discount.")
```

else:

```
print("Not eligible for a senior discount.")
```

o/p:

30% Senior discount!

conditional expression (ternary operator):

- Conditional expression (ternary operator) is a short way to write a if else statement in a single line.
- It is used when choosing between two values based on a condition.

Ex: age = 20

```
s = "Adult" if age >= 18 else "Minor"
print(s)
```

o/p: Adult

Match-Case Statement:

- Match-Case statement is used to compare a value against multiple patterns and execute the block of code.
- It is similar to switch-case statement available in other programming languages.

Ex: number = 2

```
match number:
```

```
    case 1:
```

```
        print("One")
```

```
    case 2 | 3:
```

```
        print("Two or Three")
```

```
    case _:
```

```
        print("Other number")
```

o/p: Two or Three

Multiple conditions with if statement:

- In python, we often need to make decisions based on more than one condition.

Ex-1 ÷ age = 18

if age >= 18:

print("You are an adult")

else:

print("You are a minor")

O/P ÷ You are an adult.

Ex-2 ÷ age = 18

if (age >= 8 and age <= 12):

print("You are Allowed. Welcome!")

else:

print("Sorry! You are not Allowed. BYE!")

O/P ÷ Sorry! You are not Allowed. BYE!

Ex-3 ÷ var = 'N'

if (var == 'y' or var == 'Y'):

print("You said yes")

elif (var == 'N' or var == 'n'):

print("You said No")

else:

print("Invalid Input")

O/P ÷ You said No

Ex-4 ÷ a = 7

b = 9

c = 3

if a > b and a > c:

print(a, " is the largest")

elif b > a and b > c:

print(b, " is the largest")

elif c > a and c > b:

print(c, " is the largest")

else:

print("All numbers are equal")

O/P ÷ 9 is the largest

Ex-5 ÷ a = 1

b = 1

c = 1

if (a==1 and b==1 and c==1):

Print ("Working")

else:

print ("Stopped")

O/P ÷ Working

Taking conditional user inputs:

- It means asking user for input and then checking it against certain conditions before accepting it.
- It's useful for creating interactive and error-free programs.

Ex ÷ age = int(input("Enter your age:"))

if age >= 18:

print ("Eligible")

else:

Print ("not eligible")

O/P ÷ Enter your age: 22

Eligible

- using if and else conditions

Ex ÷ num = int(input("Enter a number:"))

if num > 0:

print ("Positive Number")

else:

print ("Not Positive Number")

O/P ÷ Enter a number: 12

Positive Number

- using if, elif and else conditions

Ex ÷ marks = int(input("Enter your marks:"))

if marks >= 90:

print ("Grade A")

elif marks >= 75:

print ("Grade B")

elif marks >= 60:

print ("Grade C")

else:

print("Grade 0")

o/p: Enter your marks: 98
Grade A

using nested if conditions

ex: username = input("enter username: ")

password = input("enter password: ")

if username == "admin":

if password == "1234":

print("login successful")

else:

print("Incorrect password")

else:

print("unknown username.")

o/p: Enter username: admin

Enter Password: 1234

Login successful

Loops:

loops are used to execute a block of code repeatedly until a condition is met.

The main types of loops in python are:

1. For Loop

2. While Loop

1. For Loop

for loop is used to iterate over a sequence such as list, tuple, string or range.

It executes a block of code once for each item in the sequence.

ex: n = 4

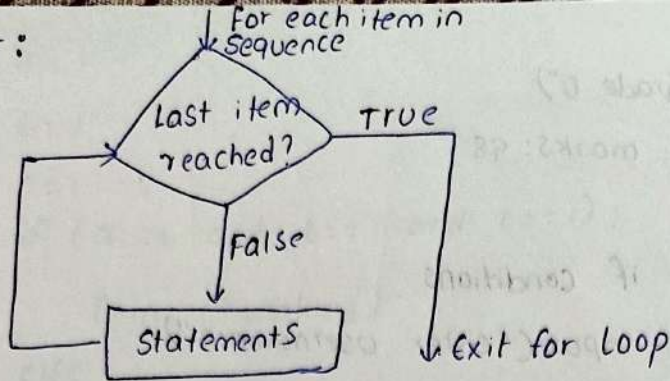
for i in range(0, n):

print(i)

o/p: 0
1
2
3

o/p: range(0, n) generates numbers from 0 to 3.

Flowchart :



Iterating by index of sequences:

- A for loop can also iterate through sequence elements using their index values with the help of range() and len().
- range() is used to generate a sequence of integers within a specified range.
- It is to control how many times a block of code runs.

Ex: for i in range(5): o/p: 0 1 2 3 4
 print(i, end=" ")

syntax: range(start, stop, step)

- start - Starting number of the sequence
- stop - Numbers at which the sequence stops
- step - Difference btw numbers (optional)

Ex: for n in range(5, 10): o/p: 5 6 7 8 9
 print(n, end=" ")

Ex: for i in range(0, 10, 2): o/p: 0 2 4 6 8
 print(i, end=" ")

Ex: for i in range(10, 0, -2): o/p: 10 8 6 4 2
 print(i, end=" ")

- len() is used to get the number of items in an object.
- It returns an integer value representing the length or the number of elements.

Ex: s = "HelloWorld"
 l = len(s) o/p: 10
 print(l)

Syntax: `len(object)`

Ex: `a = ['Alice', 'for', 'Bob', 2026]`

`print(len(a))` → It returns no. of items in a list

`b = (1, 2, 3, 4)`

`print(len(b))` → It returns no. of elements in a tuple

`c = {"name": "Alice", "age": 30, "city": "New York"}`

`print(len(c))` → It counts the number of key-value pairs

O/P: 4

4

3

Ex: `a = []`

`print(len(a))`

O/P: 0, bcz list is empty so it returns 0

Ex: `a = [10, 20, 30, 40]`

`for i in range(len(a)):`

`print("Index:", i, "Value:", a[i])`

O/P: Index: 0 Value: 10

Index: 1 Value: 20

Index: 2 Value: 30

Index: 3 Value: 40

2. While Loop

- while loop repeatedly executes a block of code as long as the given conditions remains true.
- when, the condition becomes false, the line immediately after the loop in the program is executed.

Ex: `cnt = 0`

`while (cnt < 3):`

`cnt = cnt + 1`

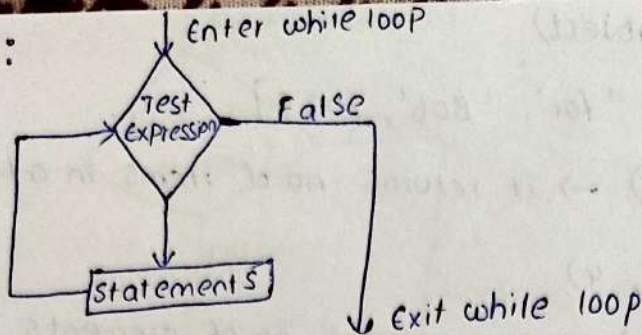
`print("Hello")`

O/P: Hello

Hello

Hello

Flowchart:



Infinite while loop

• It runs continuously because its condition always remains true.

Ex: while (True):

print("Hello")

• It is suggested not to use this type of loop as it is never ending infinite loop where the condition is always true and have to forcefully terminate the program.

Nested Loops

• A nested loop is a loop inside another loop.
• The inner executes completely for every iteration of the outer loop.

Ex: for i in range(1,5):

for j in range(i):

print(i, end=' ')

Print()

O/P:

1
2 2
3 3 3
4 4 4 4

Ex:

for i in range(2):

print(i)

for j in range(10,13):

print(j)

inner loop

outer loop

printed by inner loop

printed by outer loop

O/P

0
10
11
12
1
10
11
12

ex: $x = [1, 2]$

$y = [4, 5]$

for i in x:

for j in y:

print(i, j)

o/p:

1 4

1 5

2 4

2 5

ex: for i in range(2, 4):

for j in range(1, 11):

print(i, "*", j, "=", i * j)

print()

o/p: 2 * 1 = 2

2 * 2 = 4

2 * 3 = 6

2 * 4 = 8

2 * 5 = 10

2 * 6 = 12

2 * 7 = 14

2 * 8 = 16

2 * 9 = 18

2 * 10 = 20

3 * 1 = 3

3 * 2 = 6

3 * 3 = 9

3 * 4 = 12

3 * 5 = 15

3 * 6 = 18

3 * 7 = 21

3 * 8 = 24

3 * 9 = 27

3 * 10 = 30

ex: a1 = ['I am', 'you are']

a2 = ['healthy', 'fine', 'good']

a2_size = len(a2)

for item in a1:

print("start outer for loop")

i = 0

while (i < a2_size):

print(item, a2[i])

i = i + 1

print("end for loop")

o/p:

start outer for loop

I am healthy

I am fine

I am good

end for loop

start outer for loop

you are healthy

you are fine

you are good

end for loop

Using break statement in nested loops :

- we use break statement to exit from the loop.
- when we use a break statement in a loop it skips the rest of iteration and terminates the loop.

Ex: for i in range(10):

print(i)

if i==6:

break

o/p:

0

1

2

3

4

5

6

Ex: a=[1, 3, 5, 7, 9, 11]

val=7

for i in a:

if i==val:

print(f"Found at {i}!")

break

else:

print(f"not found")

o/p:

Found at 7!

Ex: c=5

while True:

print(c)

c-=1

if c==0:

print("Countdown finished!")

break

o/p:

5

4

3

2

1

Countdown finished!

Ex ÷ matrix = [[1, 2, 3],
[4, 5, 6],
[7, 8, 9]]

val = 5

found = False

for x in matrix:

for n in x:

if n == val:

print(f"{val} found!")

found = True

break

if found:

break

o/p ÷ 5 found!

Continue statement:

The continue statement in python is a loop control statement that skips the rest of the code inside the loop for the current iteration and moves to the next iteration immediately.

Ex ÷ for i in range(1, 11)

if i == 6:

continue

print(i, end=" ")

o/p ÷ 0 1 2 3 4 5 7 8 9 10

Ex ÷

for char in "Helloworld":

if char == 'l':

continue

print(char, end=" ")

o/p ÷ H e o l w o r d

Ex: $a = [[1, 2, 3], [4, 5, 6], [7, 8, 9]]$

for row in a:

for num in row:

if num==3:

continue

print(num, end=" ")

O/P: 1 2 4 5 6 7 8 9

Ex: $i = 0$

while $i < 10$:

if $i == 5$:

$i += 1$

continue

print(i)

$i += 1$

O/P: 0
1
2
3
4
6
7
8
9

• To ignore certain values without breaking the loop we use 'continue' statement.

Pass statement :

- It is a placeholder that does nothing when executed.
- It is used to keep code blocks valid where a statement is required but no logic is needed yet.

Ex: $x = 10$

if $x > 5$:

pass

else:

print("x is 5 or less")

O/P: No output.

when $x > 5$, the pass statement runs, so nothing happens. if $x \leq 5$, else block executes and prints the message.

Ex: for i in range(5):

if $i == 3$:

pass

else:

print(i)

o/p: 0
1
2
4

Functions :

- python functions are reusable blocks of code used to perform a specific task.
- function can be defined using def keyword.
- The def keyword is used to define user-defined functions.
- Functions help organize code into reusable blocks, making programs easier to read, maintain and reuse.
- They can accept input values through parameters, perform specific tasks and optionally returns results.

ex: def func():
 print("Hello")
func()

o/p: Hello

```
def function_name(parameters):  
    # statement  
    return expression
```

def: is a keyword to define a function

function_name: name of the function

Parameters: optional input values (can be empty)

return: optional, sends a value back from the function.

• The indented block is executed when the function is called.

ex: def sub(x, y):

 return(x-y)

a = 90

b = 50

res = sub(a, b)

Print("Subtraction of ", a, " and ", b, " is ", res)

o/p: Subtraction of 90 and
50 is 40

Ex ÷ def fun(n):

x = 2

count = 0

while count < n:

for d in range(2, int(x ** 0.5) + 1):

if x % d == 0:

break

else:

print(x)

count += 1

x += 1

n = 10

fun(n)

O/P ÷

2

3

5

7

11

13

17

19

23

29

Ex ÷ def greet(name = "Guest"):

Print("Hello", name)

O/P ÷

greet("Alice")

Hello Guest

Hello Alice

Ex ÷

Passing Function as an Argument

• Functions can be passed as an argument to another function

Ex ÷ def process(func, text):

return func(text)

def uppercase(text):

return text.upper()

print(process(uppercase, "hello"))

O/P ÷ HELLO

Higher order functions

Higher order functions play an imp role in functional programming and allow for writing more modular, reusable and readable code.

A HOF is a function that either:

- Takes another function as an argument
- Returns a function as a result

```
ex: def greet(func):  
    return func("Hello")  
def uppercase(text):  
    return text.upper()  
print(greet(uppercase))
```

o/p: HELLO

- Passing a functions as an arguments
- A function can be passed an argument to another function, enabling dynamic behaviour.

```
ex: def apply(func, x):  
    return func(x)
```

o/p: 25

```
def square(n):  
    return n*n  
print(apply(square, 5))
```

- Returning a function

```
ex: def fun(n):  
    return lambda x: x * n
```

double = fun(2)

triple = fun(3)

print(double(5))

print(triple(5))

o/p:

10

15

Lambda Functions

- Lambda function is a small anonymous function written using the lambda keyword.
- It is commonly used when passing simple functions as arguments.


```

ex: def fun(func, number):
    return func(number)

print(fun(lambda x: x ** 2, 5))

```

o/p ÷ 25

- Lambda function are small anonymous functions, means they do not have a defined name.
- These are small, short-lived functions used to pass simple logic to another function.
- Contain only one expression
- Result of that expression is returned automatically.

```

ex: a = 'HelloWorld'
upper = lambda x: x.upper()
print(upper(a))

```

o/p ÷ HELLOWORLD

- lambda is a function that takes an argument 'x' and returns x.upper()

Syntax:

```

a = lambda x: x ** 2

```

↓ ↓ ↓
 function lambda Expression
 name keyword Argument

```

ex: check = lambda x: "positive" if x > 0 else "negative" if
x < 0 else "zero"
print(check(5))
print(check(-3))
print(check(0))

```

o/p ÷ Positive
Negative
zero

Using *args

allows a function to accept a variable number of arguments which are collected into

a tuple, making the function flexible to handle multiple inputs.

In python `*args` and `**kwargs` are used to allow functions to accept an arbitrary number of arguments. ^{key word}

ex: `def fun(*args):`

`for arg in args:`

`print(arg)`

`fun(1, 2, 3, 4, 5)`

o/p:
1
2
3
4
5

ex: `**kwargs` lets a function accept any number of keyword arguments. These arguments are collected into a dictionary

ex: `def fun(**kwargs):`

`for k, val in kwargs.items():`

`print(f"{k}: {val}")`

`fun(name="Olivia", age=30, city="New York")`

o/p: `name: Olivia`

`age: 30`

`city: New York`

Here, `def fun(**kwargs):` defines a function that can take any no. of keyword arguments, stored as dictionary `kwargs`.

ex: Function Arguments

`def evenOdd(x):`

`if (x % 2 == 0):`

`return "Even"`

`else:`

`return "odd"`

`Print(evenOdd(16))`

`Print(evenOdd(7))`

o/p:

Even

odd

Types of Function Arguments :

• Python supports different types of arguments that can be passed during a function call.

① Default Arguments : It uses a predefined value when no value is passed during the function call.

Ex: `def myFun(x, y=50):`

`print("x: ", x)`

`print("y: ", y)`

`myFun(10)`

o/p: `x: 10`

`y: 50`

② Keyword Arguments : Pass values using parameter names, so argument order does not matter.

Ex: `def student(fname, lname):`

`print(fname, lname)`

`student(fname='Alice', lname='Bob')`

`student(lname='Bob', fname='Alice')`

o/p: `Alice Bob`

~~Bob~~ ~~Alice~~

`Alice Bob`

③ Positional Arguments : values are assigned to parameters based on their order in the function call.

Ex: `def nameAge(name, age):`

`print("Hi, I am", name)`

`print("My age is ", age)`

`print("Case-1:")`

`nameAge("Olivia", 27)`

`print("Case-2:")`

`nameAge(27, "Olivia")`

o/p: `Case-1:`

`Hi, I am Olivia`

`My age is 27`

`Case-2:`

`Hi, I am 27`

`My age is Olivia`

④ Arbitrary Arguments: allow functions to accept multiple values. This is done using two special symbols:

- *args - collects extra positional arguments as tuple
- **kwargs - collects extra keyword argument as a dictionary.

ex:

```
def myfunc(*args, **kwargs):  
    print("Non-keyword Arguments (*args):")  
    for arg in args:  
        print(arg)  
    print("keyword Arguments (**kwargs):")  
    for key, value in kwargs.items():  
        print(f"{key} = {value}")  
myfunc('Hey', 'Welcome', first='Alice', mid='for',  
       last='Bob')
```

O/P: Non-keyword Arguments (*args):

Hey

Welcome

keyword Arguments (**kwargs):

first = Alice

mid = for

last = Bob

Inner Functions

- A function defined another function is called inner functions (also called a nested function)
- They are mainly used for
 - Encapsulation: Hiding helper logic from external access.
 - Code Organization: Grouping related functionality for cleaner code.
 - Access to outer variable: Inner functions can use variable of the outer function.
 - Closures and Decorators: supporting advanced features like closures (functions that remember values) and decorators.

Ex-1: def fun1(msg): //outer function

def fun2(): //inner function

print(msg)

fun2()

O/P: Hello

fun1("Hello")

ex-2: def fun1():

msg = "Hello world"

def fun2():

print(msg)

O/P: Hello world

fun2()

fun1()

• Here fun1() creates a local variable msg and defines fun2(), which prints msg. Because of lexical scoping, fun2() can access msg from fun1() and calling fun1() runs fun2() and prints the msg.

ex-3: def fun1():

a = 45

def fun2():

nonlocal a

O/P: 54

a = 54

print(a)

fun2()

print(a)

fun1()

ex-4: def fun1(a):

def fun2():

print(a)

return fun2

closure_func = fun1("Hello, Closure!")

closure_func()

O/P: Hello, Closure!

• Even after fun1() finishes, fun2() still remembers the value of a. This behaviour is called a closure.

Return Statement

- Return is used to end a function and send a value back to the caller.
- It can return any datatype, multiple values (packed into a tuple), or None if no value is given.

Syntax: `return [Expression]`

- Once return is executed, the function stops the running and any code written after it is ignored. If no value is returned, Python automatically returns None.

Ex: `def Square(n):` O/P: 16
 `return n*n;`
 `res = square(4)`
 `print(res)`

Syntax: `def function_name(parameters):`
 #function body
 return value

Ex: Returning multiple values

`def get_vals():`

`x = 10`

`y = 20`

`return x, y`

`a, b = get_vals()`

`print(a)`

`print(b)`

O/P: 10
20

Ex: `def calc(n):`

`return [n*n, n*n, n*n]`

O/P: [9, 27]

`print(calc(3))`

Returning a list from a function

Ex: Function Returning Another function

`def outer(msg):`

O/P: Hello

`def inner():`

`return msg`

`return inner`

`print(outer("Hello"))`

Ex ÷ def sq_value(num):

return num*2

O/P ÷

Print(sq_value(2))

4

Print(sq_value(4))

16

Pass by Reference and pass by value ÷

- variables refer to objects.
- Function behaviour depends on whether the object is mutable or immutable.
- mutable objects like list can be modified inside functions.
- Immutable objects like integers and strings remain unchanged.

Ex ÷ def myfun(x):

x[0] = 20

b = [10, 11, 12, 13]

myfun(b)

print(b)

def myfun2(x):

x = 20

a = 10

myfun2(a)

print(a)

O/P ÷

[20, 11, 12, 13]

10

Python Built-in Functions List ÷

Function Name

Description

Python abs(). Function

Return the absolute value of a number

Python iter(). Function

It takes an asynchronous iterable as an argument and returns asynchronous iterator for that iterable

Python all(). Function

Returns true if all the elements of a given iterable

Python any(). Function

Returns true if any of the elements of a given iterable are true else it returns false

Python next(). Function

used for getting the next item from an asynchronous iterator

Python ascii(). Function

Returns a string containing containing a printable representation of an object

Python dict(). Function

creates a Python Dictionary

python bin(). Function
python bool(). Function
python breakpoint().
Functions

convert integer to a binary string
Return or convert a value to a Boolean value i.e., true or false
it is used for dropping into the debugger at the call site during runtime for debugging process.

python bytearray().
Function

Returns a byte array object which is an array of given bytes.

python callable().
Function

Returns True if the object passed appears to be callable.

python chr(). Function

Returns a string representing a character whose unicode code point is an integer.

python compile().
Function

Returns a python code object.

python complex().
Function

creates complex number

python delattr().
Function

Delete the named attribute from the object

python dict(). Function

creates a python dictionary

python dir(). Function

Returns a list of the attributes and method of any object.

python divmod().
Function

Take two numbers and returns a pair of number consisting of their quotient and remainder

python eval(). Function

parses the expression passed to it and runs python expression within the program.

python exec(). Function

used for dynamic execution of the program.

python hash(). Function

encode the data into an unrecognizable value

python help(). Function

Display the documentation of modules, functions, classes, keywords etc.,

python id(). Function

Return the identity of an object.

python int(). Function

converts a number in a given base to decimal

python len(). Function

Returns the length of the object
creates a list in python

python list(). Function

python map(). Function

Returns a map object of the results after applying the given function to each item of a given iterable.

Python max().function

Python min().function

Python next().function

Python object().function

Python range().function

Python set().function

Python str().function

Python sum().function

Python super().function

Python type().function

Python _import_().function

Python slice().function

Python print().function

Python oct().function

Returns the largest item in an iterable.

Returns the smallest item in an iterable.

Receives the next item from the iterator.

Returns a new object.

Generate a sequence of numbers.

convert any of the iterable to a sequence of iterable

Returns the string version of the object

sums up the numbers in the list

Returns a temporary object of the superclass

Returns the type of the object

imports the module during runtime

Returns a slice object

print output to the console.

Returns an octal representation of an integer in a string format.

Recursion :

• Recursion is a programming technique where a function calls itself either directly or indirectly to solve a problem.

• It is commonly used for:

→ Breaking Problems into smaller subproblems

→ Mathematical calculations like factorial and fibonacci

→ Tree and graph traversal and Divide-and-Conquer algorithms.

Working :

A recursive function calls itself in its body.

```
def recursive-function(parameters):
```

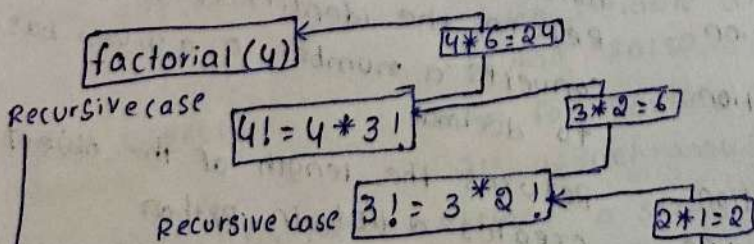
```
    if base-case-condition:
```

```
        return base-result
```

```
    else :
```

```
        return
```

```
        recursive-function(modified-parameters)
```



Part of the function where it call itself with modified parameters.

Base case

↓
stopping condition that prevents infinite recursion.

ex: def factorial(n):
 if n==0:
 return 1
 else:
 return n * factorial(n-1)
 print(factorial(5))

calculate factorial of a number case

O/P: 120

ex: calculate nth Fibonacci number.

def fibonacci(n):
 if n==0:
 return 0
 elif n==1:
 return 1
 else:
 return fibonacci(n-1) + fibonacci(n-2)
 print(fibonacci(10))

O/P: 55

PYTHON DATA STRUCTURES

String:

- Strings are sequence of characters written inside quotes.
- It can include letters, numbers, symbols and spaces.
- Python does not have a separate character type.
- A single character is treated as a string of length one.
- Strings are commonly used for handling and manipulation.

creating a string

- String can be created using either single or double ('') or ("") quotes. Both behave the same.

ex: a = 'SFS'
 b = "AliceForBob"
 print(a)
 print(b)

O/P: SFS
 AliceForBob

Multi-line String

- use triple quotes ('''...''') or (""""...""") for strings that span multiple lines. Newlines are preserved.

ex: s = """ I am Learning Python """
 print(s)
 t = ''' I am a Student '''
 print(t)

O/P:
 I am Learning Python
 I am a student

Accessing characters in String

- Strings are indexed sequences.
- positive indices start at 0 from the left.
- Negative indices start at -1 from the right.

0	1	2	3	4	5	6	7	8	9	10	11	12	13
P	Y	T	H	O	N	L	E	A	R	N	I	N	G
-14	-13	-12	-11	-10	-9	-8	-7	-6	-5	-4	-3	-2	-1

Ex: $S = \text{"ABCDEF"}$ o/p:
 $\text{print}(S[0])$ A
 $\text{print}(S[4])$ E

- Accessing an index out of range will cause an `IndexError`.
- Only integers are allowed as indices and using a float or other types will result in a `TypeError`.

String slicing

- It is a way to get specific parts of a string by using start, end and step values.
- It's especially useful for text manipulation and data parsing.

Syntax: $\text{substring} = S[\text{start}:\text{end}:\text{step}]$

Ex: $S = \text{"Hello, Python"}$ o/p: Hello
 $\text{print}(S[0:5])$

characters using negative indices

Ex: $S = \text{"abcdefghijklmno"}$ o/p: lmno
 $\text{print}(S[-4:])$ abcdefghijkl
 $\text{print}(S[-3])$ klm
 $\text{print}(S[-5:-2])$ hijl
 $\text{print}(S[-8:-1:2])$

Reverse a string using slicing

- To reverse a string, use a negative step value of -1.

Ex: $S = \text{"Python"}$ o/p: nohtYP
 $\text{print}(S[::-1])$

Ex: $S = \text{"Hello, World!"}$ o/p: $\longrightarrow$ Retrieve All characters
 $S2 = S[:]$ Hello, World!
 $S3 = S[::-1]$ Hello, world!
 $\text{print}(S2)$
 $\text{print}(S3)$

• using `[:]` or `[::]` without specifying start, end or step returns the complete string.

Ex-2 ÷ Get all characters Before or After specific position

```
s = "Hello, World!"    o/p ÷  
print(s[7:])           World!  
print(s[:5])           Hello
```

Ex-3 ÷ Extract characters Between two positions

```
s = "Hello, World!"    o/p ÷  
print(s[1:5])          ello
```

Ex-4 ÷ Get characters at specific intervals

```
s = "abcdefghi"        o/p ÷  
print(s[:2])           acegi  
print(s[1:8:3])         beh
```

Looping Through Strings

• Strings are iterable, which means we can access each character one by one using a loop.

```
ex ÷ s = "ABCDEF"      o/p ÷ A  
for char in s:         B  
    print(char)        C  
                        D  
                        E  
                        F
```

String Immutability

• Strings are immutable, means their values cannot be changed after creation.
• Any modification to a string creates a new string instead of altering the original one.

```
ex ÷ s = "aBCDEF"      o/p ÷ ABCDEF  
s = "A" + s[1:]  
print(s)
```

Deleting a string

• Here, strings cannot be deleted because strings are immutable.

• After deleting a string, if we try to access it, python raises a `NameError` because the variable no longer exists in memory.

Updating a String

- strings cannot be changed directly after creation.
- so any modification results in a new string being created using slicing or methods like `replace()`.

ex: `s = "ABCD EF"`

`s1 = "H" + s[1:]`

`s2 = s.replace("ABC", "abc")`

o/p: `HB CD EF`

`abcd EF`

Common String Methods

1. `len()`: returns the total number of characters in a string

ex: `s = "HelloWorld"`

`print(len(s))`

o/p: `10`

2. `upper()` and `lower()`: `upper` method converts all lowercase characters to upper case characters.

ex: `s = "Hello World"`

`print(s.upper())`

`print(s.lower())`

o/p: `HELLO WORLD`

`hello world`

3. `strip()` and `replace()`: `strip()` removes leading and trailing whitespaces from the string.

`replace()` replaces all occurrences of a specified substring with another.

ex: `s = " ABC "`

`print(s.strip())`

`s = "python is fun"`

`print(s.replace("fun", "awesome"))`

o/p: `ABC`

`python is awesome`

4. `title()`: converts string to title case.

ex: `s = "helloworld"`

`print(s.title())`

o/p: `Helloworld`

ex: `s = "hello world"`

`print(s.title())`

o/p: `Hello World`

5. `swapcase()`: swap the cases of all characters in a string used to return a new string where all the uppercase characters in the original string are converted to lowercase and all the lowercase characters are converted to uppercase.

ex: `s = "I love python"`

`print(s.swapcase())`

o/p: `i LOVE PYTHON`

6. `capitalize()`: convert the 1st character of a string to uppercase and automatically convert all remaining characters to lowercase.

ex: `s = "hello world"`

`print(s.capitalize())`

o/p: `Hello world`

7. `count()`: Returns the number of occurrences of a substring in the string.

ex: `s = "hello World"`

`print(s.count(o))`

o/p: `2`

Syntax: `string.count(substring, start=0, end=len(s))`

ex: `s = "apple banana apple grape apple"`

`substring = "apple"`

`print(s.count(substring, 1, 20))`

o/p: `1`

8. `endswith()`: Returns true if the given string ends with the given suffix.

ex: `s = "Hello World"`

`print(s.endswith("world"))`

o/p: `True`

Syntax: `string.endswith(suffix, start, end)`

↓
nothing but a string that needs to be checked

ex: `s = "Hello World"`

`print(s.endswith("World", 4, 10))`

o/p: `True`

9. `find()`: Returns the lowest index of the substring if it is found.

if the substring is not found it returns -1. It is case-sensitive.

Ex: `s="Welcome to Pythonclass"`
`Print(s.find("Pythonclass"))`

o/p: 11

• Here space is also included.

Ex: `s="Python is fun"`
`print(s.find("python"))`

o/p: -1

Ex: `s="abc abc abc"`
`print(s.find("abc", 4))`

o/p: 4

10) `index()`: used to find the position of a specified substring within a given string.

• It is similar to `find()` method but raises a `ValueError` if the substring is not found.

Ex: `s="Python programming"`
`print(s.index("prog"))`

o/p: 7

• It returns the starting index.

Ex: `s="python programming is fun"`
`print(s.index("is", 10, 25))`

o/p: 19

• `start=10`, `end=25`, so `index()` searches btw these positions.

11) `isalnum()`: checks whether all the characters in a given string is alphanumeric or not.

Ex: `s="Python123"`
`Print(s.isalnum())`

o/p:

True

12) `isalpha()`: checks whether all the characters in given string is alphabet or not.

Ex: `s1="HelloWorld"`

`s2="Hello123"`

`print(s1.isalpha())`

`print(s2.isalpha())`

Ex: `s="Python!"`

`print(s.isalpha())`

o/p: False

o/p: True

False

• special characters allowed, it returns false.

• It is purely alphabetic.

13) `isdecimal()`: Returns true if all characters in a string are decimal.

ex: `s1 = "12345"` o/p: `True`
`s2 = "12345a"` `False`
`s3 = "123.45"` `False`
`s4 = "-12345"` `False`

`print(s1.isdecimal())`

`print(s2.isdecimal())`

`print(s3.isdecimal())`

`print(s4.isdecimal())`

14) `isdigit()`: Returns true if all characters in a string are digits.

ex: `a = "12345"` o/p: `True`
`b = "12345a"` `False`
`print(a.isdigit())`
`print(b.isdigit())`

15) `isidentifier()`: checks whether a string is valid identifier or not.

ex: `s = "variable.name"`
`print(s.isidentifier())`
o/p: `True`

16) `islower()`: checks if all characters in the string are lowercase.

ex: `s = "hello"`
`print(s.islower())`
o/p: `True`

17) `isnumeric()`: Returns true if all characters in the string are numeric characters.

ex: `s = "12345678"`
`print(s.isnumeric())`
o/p: `True`

18) `join()`: Returns a concatenated string.

ex: `t = ("Learn", "python", "Fast")`
`print("-".join(t))`

o/p: `Learn-python-Fast`

19) `translate()`: used to transform a string by replacing or removing specific characters based on translation table.

ex: `tab = str.maketrans("aeiou", "12345")`
`s = "hello world"`
`print(s.translate(tab))` o/p: `h2ll4'w4rld`

Lists :

COLLECTIONS

- List is a built-in data structure used to store an ordered collection of items.
- They are dynamic, resizable and capable of storing multiple data types.
- Mutable means list elements can be changed, updated, added or removed after the list is created.
- ordered means elements maintain the order in which they are inserted.
- Index-based means elements are accessed using their position, starting from index 0.

Creating a list

- List can be created in several ways, such as using square brackets [], the list() constructor or by repeating elements.
1. Using square Brackets: Square brackets [] are used to create a list directly.

ex: `a = [1, 2, 3]`

`print(a)`

`b = ["apple", "banana"]`

`print(b)`

O/P: `[1, 2, 3]`

`['apple', 'banana']`

2. Using list() constructor: A list can also be created by passing an iterable to the list() constructor.

ex: `a = list([1, 2, 3, 'apple', 4.5])`

`print(a)`

`b = list("GFG")`

`Print(b)`

O/P: `[1, 2, 3, 'apple', 4.5]`

`['G', 'F', 'G']`

3. Creating List with Repeated Elements: A list with repeated elements can be created using the multiplication operator.

ex: `a = [2] * 5`

O/P:

`[2, 2, 2, 2, 2]`

`b = [0] * 7`

`print(a)`

`print(b)`

`[0, 0, 0, 0, 0, 0, 0]`

Internal representation of Lists

- Python list stores references to objects, not the actual values directly.
- The list ^{keeps} memory addresses of objects like integers, strings, or booleans.
- Actual objects exist separately in memory.
- Modifying a mutable object inside a list changes the original object.

• Reassigning an immutable object creates a new object instead of changing the old one.

Ex ÷ `a = [1, 2, 2, "python"]` o/p ÷ 1
 `print(a[0])` `[1, 2, 2, 'python']`
 `print(a)`

Accessing List Elements

• Elements in list are accessed using indexing.

• Python uses zero-based indexing, means `a[0]` represents 1st element.

• Negative indexing is also supported, where `-1` accesses the last element.

Ex ÷ `a = [10, 20, 30]` o/p ÷ 10
 `print(a[0])` 30
 `print(a[-1])`

Adding Elements into List

• Elements can be added to a list using the following methods.

1. `append()` ÷ Adds an element at the end of the list.

Ex ÷ `a = [1, 2]` o/p ÷ `[1, 2, 3]`
 `a.append(3)`
 `print(a)`

2. `insert()` ÷ Adds an element at a specific position.

Ex ÷ `a = [1, 3]` o/p ÷ `[1, 2, 3]`
 `a.insert(1, 2)`
 `print(a)`

3. `extend()` ÷ Adds multiple elements to the end of the list.

Ex ÷ `a = [1, 2]` o/p ÷ `[1, 2, 3, 4]`
 `a.extend([3, 4])`
 `print(a)`

Updating Elements into List

• Since lists are mutable, elements can be updated by assigning new values using their index.

Ex: $a = [10, 20, 30, 40, 50]$ o/p: $[10, 25, 30, 40, 50]$

$a[1] = 25$

$\text{print}(a)$

Removing Elements from List

• Elements can be removed from a list using the following methods.

1. $\text{remove}()$ ÷ Removes the first occurrence of an element.

Ex: $a = [1, 2, 3]$ o/p: $[1, 3]$

$a.\text{remove}(2)$

$\text{print}(a)$

2. $\text{pop}()$ ÷ Removes the element at a specific index or the last element if no index is specified.

Ex: $a = [1, 2, 3]$ o/p: $[1, 2]$

$a.\text{pop}()$

$\text{print}(a)$

3. del statement ÷ Deletes an element at a specified index.

Ex: $a = [1, 2, 3]$ o/p: $[1, 3]$

$\text{del } a[1]$

$\text{print}(a)$

4. $\text{clear}()$ ÷ removes all items.

Ex: $a = [1, 2, 3]$ o/p: $[]$

$a.\text{clear}()$

$\text{print}(a)$

Iterating over Lists

• Lists can be iterated using loops, allowing operations to be performed on each element.

Ex: $a = ['apple', 'banana', 'cherry']$ o/p: apple

for item in a:

banana

cherry

$\text{print}(a)$

Nested Lists

• A nested list is a list that contains another list as its element.

• It is commonly used to represent matrices or tabular data.

• nested elements can be accessed by chaining multiple indexes.

Ex ÷ $a = [[1, 2], [3, 4]]$ o/p ÷ $[1, 2]$

$\text{print}(a[0])$

3

$\text{print}(a[1][0])$

• Time complexity of accessing an element in a list using its index is $O(1)$

Tuples :

- A tuple is an immutable ordered collection of elements.
- Tuples are similar to lists, but unlike lists, they cannot be changed after their creation.
- Can hold elements of different data types.
- These are ordered, heterogeneous and immutable.

Creating a tuple

• A tuple is created by placing all the items inside parentheses(), separated by commas. A tuple can have any no. of items.

Ex ÷ ① $\text{tup} = ()$

$\text{print}(\text{tup})$

② $\text{tup} = ('Great', 'For')$

$\text{print}(\text{tup})$

③ $\text{tup} = \text{tuple}('Great')$

$\text{print}(\text{tup})$

④ $li = [1, 2, 4, 5, 6]$

$\text{print}(\text{tuple}(li))$

O/P ÷ $()$

$('Great', 'For')$

$('G', 'r', 'e', 'a', 't')$

$(1, 2, 4, 5, 6)$

Creating a tuple with mixed DataTypes :

• Tuple can store elements of different data types such as integers, strings, lists and dictionaries, within a single structure.

Ex ÷ $\text{tup} = (5, 'Welcome', 'True', [1, 2, 3], \{'key': 'value'\})$

$\text{print}(\text{tup})$

O/P ÷ $(5, 'Welcome', 'True', [1, 2, 3], \{'key': 'value'\})$

Tuple Basic operations :

1. Accessing of Tuples : We can access elements of a tuple by using indexing and slicing, similar to how we elements in a list.
 - Indexing starts at 0 for the first elements and goes up to $n-1$, where n is the number of elements in a tuple.
 - Negative indexing starts from -1 for the last element and goes backward.

ex ÷ `tup = tuple("Great")` o/p ÷
`print(tup[0])` G
`print(tup[1:4])` ('r', 'e', 'a')
`print(tup[:3])` ('G', 'r', 'e')

ex ÷ `tup = ("Alice", "For", "Bob")`
`a, b, c = tup` o/p ÷
`print(a)` Alice
`print(b)` For
`print(c)` Bob

2. Concatenation of Tuples : Tuples can be concatenated using the "+" operator.
 - This operation combines two or more tuples to create a new tuple.
 - Only tuples can be concatenated with tuples.
 - Combining a tuple with other types like lists will raise an error.

ex ÷ `tup1 = (0, 1, 2, 3)` o/p ÷
`tup2 = ('Alice', 'For', 'Bob')` (0, 1, 2, 3, 'Alice', 'For', 'Bob')
`tup3 = tup1 + tup2`
`print(tup3)`

3. Slicing of a tuple : It means creating a new tuple from a subset of elements of the original tuple.

- The slicing syntax is :

`tuple[start : stop : step]`

ex ÷ `tup = tuple('ALICEFORBOB')`
`print(tup[1:])`
`print(tup[::-1])`
`print(tup[4:9])`

o/p ÷ ('L', 'I', 'C', 'E', 'F', 'O', 'R', 'B', 'O', 'B')

('B', 'O', 'B', 'R', 'O', 'F', 'E', 'C', 'I', 'L', 'A')

('E', 'F', 'O', 'R', 'B')

3. Deleting a Tuple ÷ Since tuples are immutable, we cannot delete individual elements of a tuple. However, we can delete an entire tuple using del statement.
- printing of tuple after deletion results in an error.

ex ÷ tup = (0, 1, 2, 3, 4)

del tup

print(tup)

o/p ÷ ERROR!

Tuple unpacking with Asterisk(*)

- * is used in tuple unpacking to grab multiple items into a list.
- This is useful to extract just a few specific elements and collect the rest together.

ex ÷ tup = (1, 2, 3, 4, 5)

o/p ÷

a, *b, c = tup

[2, 3, 4]

print(a)

5

print(b)

print(c)

- a gets the first item, c gets the last item and *b collects everything in btw into a list.

Reversing a Tuple

- Most common and pythonic way to reverse a tuple is by using slicing with a step of -1.

① ex ÷ t = (1, 2, 3, 4, 5)

rev = t[::-1]

print(rev)

o/p ÷ (5, 4, 3, 2, 1)

② using reversed()

- reversed() function returns an iterator that can be converted into a tuple.

Ex: $t = (1, 2, 3, 4, 5)$

```
rev = tuple(reversed(t))
```

print(rev) $\rightarrow$ converts reversed iterator into a new tuple without modifying the original tuple.

o/p: $(5, 4, 3, 2, 1)$

③ Using a Loop

• We can manually reverse the tuple by iterating from the end using a loop.

Ex: $t = (1, 2, 3, 4, 5)$

```
rev = tuple(t[i] for i in range(len(t)-1, -1, -1))
```

```
print(rev)
```

o/p: $(5, 4, 3, 2, 1)$

• Here, $\text{range}(\text{len}(t) - 1, -1, -1)$ generates indices from the last element to the first, allowing elements access in reverse order.

*Sort() is not available for tuples.

Dictionary:

• Dictionary is a data structure that stores information in key-value pairs.

• While keys must be unique and immutable.

• Values can be any data type, whether mutable or immutable.

Ex: $\text{data} = \{ \text{"name": "Tom", "age": 22} \}$

```
print(data)
```

o/p: $\{ \text{'name': 'Tom', 'age': 22} \}$

[Note: I use double quotes (" ") in i/p, but Python prints single quote (' '). Because Python internally chooses a standard representation of string when displaying containers like dictionaries, lists and tuple. By default it uses single quotes in the output of print() for these objects.]

Creating a dictionary

• A dictionary is created by writing key value pairs inside $\{ \}$, where each ^{key} value is connected to a value using $(:)$.

• A dictionary can also be created by using the dict() function.


```
ex ÷ a = {"x": 1, "y": 2}
```

```
print(a)
```

```
b = dict(name="Sam", age=20)
```

```
print(b)
```

```
o/p ÷ {'x': 1, 'y': 2}
```

```
{'name': 'sam', 'age': 20}
```

Accessing Dictionary Items

- A value in dictionary is accessed by using its key.
- This can be done with either square brackets [] or get() method.
- Both return the value linked to the given key.

```
ex ÷ d = {"name": "kat", "age": 10}
```

```
print(d["name"])
```

```
print(d.get("age"))
```

```
o/p ÷ kat
```

```
10
```

- Accessing a missing key with [] raises a keyerror.
- while, get() is safer because it returns None instead of an error. (default)

Adding and updating Dictionary Items

- New items are added to a dictionary using the assignment operator (=) by giving a new key value.
- If an existing key is used with the assignment operator its value is updated with the new one.

```
ex ÷ d = {"name": "Sam"}
```

```
d["age"] = 21 // Adding a new key-value pair
```

```
d["name"] = "Alex" // updating an existing value
```

```
print(d)
```

```
o/p ÷ {'name': 'Alex', 'age': 21}
```

Removing Dictionary Items

- Dictionary items can be removed using built-in deletion methods that work on keys:

1. del : removes an item using its key.

Ex: $d = \{ "a" : 1, "b" : 2 \}$

del d["a"]

O/P: $\{ 'b' : 2 \}$

print(d)

2. pop(): removes the item with the given key and returns its value.

Ex: $d = \{ "a" : 1, "b" : 2 \}$

val = d.pop("a")

O/P: 1

print(val)

print(d)

$\{ 'b' : 2 \}$

3. popitem(): removes and returns the last inserted key-value pair.

Ex: $d = \{ "a" : 1, "b" : 2 \}$

O/P: $('b' , 2)$

print(d.popitem())

4. clear(): removes all items from the dictionary.

Ex: $d = \{ "a" : 1, "b" : 2 \}$

O/P: $\{ \}$

print(d.clear())

Note: popitem() returns a tuple.

Iterating Through a Dictionary

• A dictionary can be traversed using a for loop to access its keys, values or both key-value pairs by using the built-in methods keys(), values and items().

1. Iterate keys: Returns all keys from the dictionary.

Ex: $d = \{ "a" : 1, "b" : 2 \}$

O/P: a
b

for key in d:

print(key)

2. Iterate values: Returns all the values from the dictionary

Ex: $d = \{ "a" : 1, "b" : 2 \}$

O/P: 1
2

for value in d.values():

print(value)

3. Iterate key-value pairs: Returns all key-value pairs as tuples.

Ex: $d = \{ "a" : 1, "b" : 2 \}$

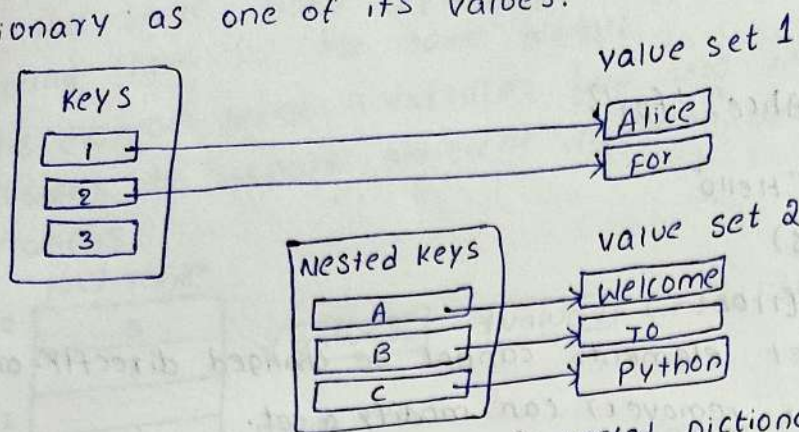
O/P: a 1
b 2

for key, value in d.items():

print(key, value)

Nested Dictionaries

A nested dictionary is a dictionary that contains another dictionary as one of its values.



This is the representation of nested dictionary.

```
Ex ÷ d = {
    "student": {
        "name": "sam",
        "age": 20
    }
}
print(d["student"]["name"])
```

o/p ÷ sam

Sets:

- Set is a built-in python data types used to store a collection of unique items.
- Stores only unique elements; duplicate values are automatically removed.
- Unordered collection, so elements do not have a fixed position and cannot be accessed using indexes.
- Supports fast search, insertion and deletion operations using hashing internally.

```
Ex ÷ S = {10, 50, 20}    o/p ÷ {10, 50, 20}
print(s)                <class 'set'>
print(type(s))
```

- There is no specific order for set elements to be printed.

TYPE Casting:

- set() method is used to convert other data types, such as lists or tuples into sets.

```
Ex ÷ s = set(["a", "b", "c"])    o/p ÷ {'c', 'b', 'a'}
print(s)
```


- Sets cannot have duplicate values.

Ex: $s = \{ "Alice", "for", "Alice" \}$

print(s)

o/p: $\{ 'Alice', 'for' \}$

Ex: $s[1] = "Hello"$

print(s)

o/p: TypeError:

bcz set elements cannot be changed directly. only add() or remove() can modify a set.

Heterogenous Element

- sets can store heterogenous elements in it, i.e., a set can store a mixture of string, integer, boolean, etc. datatypes.

Ex: $s = \{ "Alice", "for", 10, 52.7, true \}$

Print(s)

o/p: $\{ true, 52.7, 'Alice', 10, 'for' \}$

Frozen Sets

- Frozen set is an immutable version of a set. Its elements cannot be changed after creation, but you can perform operations like union, intersection and difference.

Ex: $s = \text{set}(["a", "b", "c"])$

print("Normal set:", s)

fs = frozenset(["e", "f", "g"])

print("Frozen Set:", fs)

o/p: ("Normal set:", set(['a', 'c', 'b']))

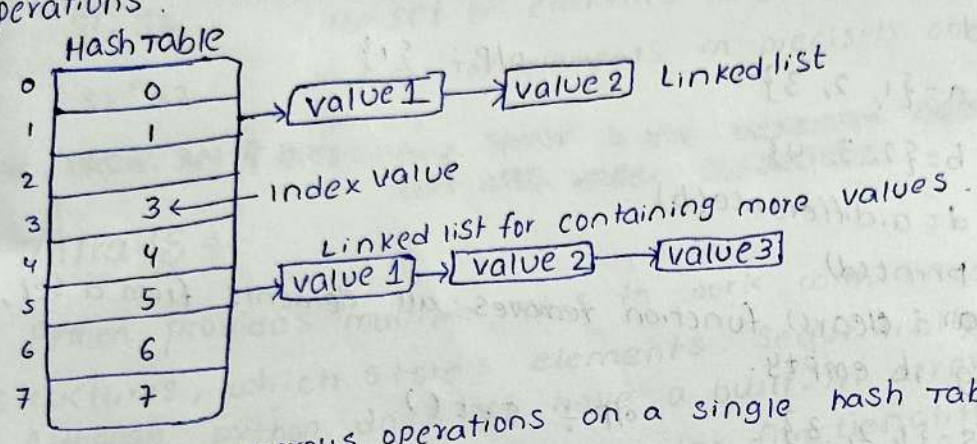
("Frozen set:", frozenset(['e', 'g', 'f']))

- frozensets are immutable, so methods like add() or remove() cannot be used.
- They are also hashable, which allows them to be used as dictionary keys.

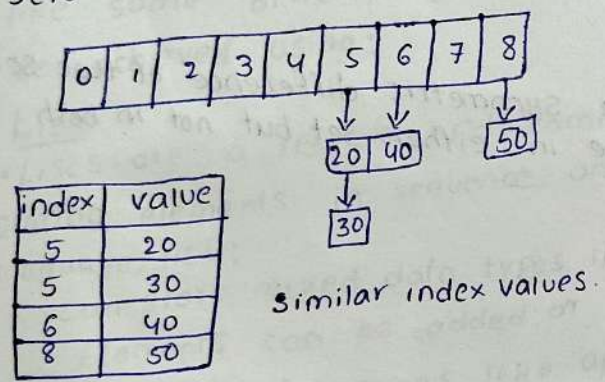
Internal working of a set

- python sets are implemented using a hash table, similar

to dictionaries, where the set elements are stored keys with dummy values. If multiple elements map to the same index, python handles the collision by storing them in the same bucket. The diagram below illustrates how sets internally manage collisions to support efficient insertion, deletion and lookup operations.



• Sets with numerous operations on a single hash table.



Methods for Sets :

- 1. Adding elements:** `add()` function is used to insert new elements into a set. it automatically ignores duplicates.

Ex: $S = \{ "a", "b", "c" \}$ o/p: $\{ 'c', 'd', 'a', 'b' \}$ (or)

```

s.add("d")
print(S)

```

$\{ 'a', 'b', 'c', 'd' \}$ (or)

$\{ 'd', 'a', 'c', 'b' \}$ (or) $\{ 'b', 'd', 'a', 'c' \}$
- 2. Union:** `union()` function combines two sets and returns a new set with all unique elements.

Ex: $a = \{ "x", "y" \}$ o/p: $\{ 'z', 'y', 'x' \}$ (or)

$b = \{ "y", "z" \}$ $\{ 'x', 'y', 'z' \}$

```

u = a.union(b)
print(u)

```
- 3. Intersection:** `intersect` function returns a new set containing elements that are common to both sets.

Ex: $a = \{1, 2, 3\}$

o/p: $\{2, 3\}$

$b = \{2, 3, 4\}$

$i = a.intersection(b)$

print(i)

4. Difference: difference() function returns a set containing elements that are in the first set but not in the second.

Ex: $a = \{1, 2, 3\}$

o/p: $\{1\}$

$b = \{2, 3, 4\}$

$d = a.difference(b)$

print(d)

5. clear: clear() function removes all elements from a set, leaving it empty.

Ex: $s = \{1, 2, 3\}$

o/p: set()

s.clear()

print(s)

6. Symmetric difference: The symmetric difference of two sets includes elements that are in either set but not in both.

Ex: $A = \{1, 2, 3, 4\}$

$B = \{3, 4, 5, 6\}$

res1 = $A \Delta B$

print("using '^':", res1)

o/p: using '^': $\{1, 2, 5, 6\}$

continuation →

res2 = A.symmetric_difference(B)

print("using symmetric_difference():", res2)

o/p: using symmetric_difference(): $\{1, 2, 5, 6\}$

Operators for sets:

sets and frozen sets support the following operators:

operators

key in s

key not in

$s1 == s2$

$s1 != s2$

$s1 < s2$

Notes

containment check

non-containment check

$s1$ is equivalent to $s2$

$s1$ is not equivalent to $s2$

$s1$ is subset of $s2$

$S1 < S2$	$S1$ is proper subset of $S2$
$S1 \supseteq S2$	$S1$ is superset of $S2$
$S1 > S2$	$S1$ is proper superset of $S2$
$S1 \cup S2$	the union of $S1$ and $S2$
$S1 \& S2$	the intersection of $S1$ and $S2$
$S1 - S2$	the set of elements in $S1$ but not $S2$
$S1 \wedge S2$	The set of elements in precisely one of $S1$ or $S2$

And, Python, set is a homogeneous data type and each element is immutable.

Arrays ÷

- Python provides multiple ways to work with linear data structures, which stores elements sequentially.
- Although python does not have a built-in array type like some other languages, similar functionality can be achieved using:

List

- Lists are a flexible and commonly used datastructure for storing elements in sequence. Unlike arrays in other language, list:
 - Can store mixed data types in one list.
 - Elements can be added or removed easily.
 - provide's functions like `append()`, `remove()`, `sort()` etc.

ex: `a = [1, "Hello", [3.14, "World"]]`

`a.append(2)`

`print(a)`

o/p ÷ `[1, 'Hello', [3.14, 'World'], 2]`

Numpy Arrays

- Numpy arrays are a part of Numpy.
- library, which is a tool for numerical computing.
- Designed for high performance operations on large datasets and support multi-dimensional arrays and matrices, making them suitable for complex mathematical computations.
- Multi-dimensional support : can handle multiple dimensions, making them suitable for matrix operations and advanced mathematical tasks.

> Broad broadcasting capabilities: Allow operations of different shapes and sizes using broadcasting.
 > Efficient storage and processing: uses optimized memory and provides faster performance compared to lists for numerical operations.

```
ex ÷ import numpy as np
a = np.array([1, 2, 3, 4])
print(a * 2)
res = np.array([[1, 2], [3, 4]])
print(res * 2)
```

o/p ÷ [2. 4. 6. 8]
 [2, 4]
 [6, 8]

Arrays

- Array is a collection of elements stored at contiguous memory locations, used to hold multiple values of the same data type.
- unlike lists, which can store mixed types, arrays are homogenous and require a typecode during initialization to define the data type.

```
ex ÷ import array as arr
a = arr.array('i', [1, 2, 3])
print(a[0])
a.append(5)
print(a)
```

o/p ÷ 1
 array('i', [1, 2, 3, 5])

→ array('i', [1, 2, 3]), i means the array will store integers.

TYPEcode	C TYPE	Python TYPE	Minimum size (Bytes)
'b'	signed char	int	1
'i'	signed int	int	4
'f'	float	float	4
'd'	double	float	8

creating Arrays

- Array can be created by importing an array module.
- array(data_type, value_list) used to create array with data type and value list specified in its arguments.


```

ex: import array as arr
a = arr.array('i', [1, 2, 3])
for i in range(0, 3):
    print(a[i], end=" ")

```

O/P: 1 2 3

Adding Elements

Elements can be added to an array using `insert()` to place value at a specific index, or `append()` to add value at the end.

```

ex: import array as arr
a = arr.array('i', [1, 2, 3])
print(*a)
a.insert(1, 4)
print(*a)

```

O/P: 1 2 3
1 4 2 3

Accessing Items

Array elements are accessed using their index with square brackets `[]`.

Each item has a position starting 0 and the index must be an integer.

```

ex: import array as arr
a = arr.array('i', [1, 2, 3, 4, 5, 6])
print(a[0])
print(a[3])
b = arr.array('d', [2.5, 3.2, 3.3])
print(b[1])
print(b[2])

```

O/P:

1
4

3.2

3.3

Removing Elements

Elements can be removed using `remove()`, which deletes the first occurrence of value or `pop()`, which removes and returns an element.

```

ex: import array
a = array.array('i', [1, 2, 3, 1, 5])
a.remove(1)
print(a)
a.pop(2)

```

O/P:

array('i', [2, 3, 1, 5])
array('i', [2, 3, 5])

slicing

- slicing is used to access a specific range of elements from an array using index positions.
- Elements from beginning to a range use `[ : index ]`
- Elements from end use `[ : -index ]`
- Elements from a specific index till the end use `[ index : ]`
- Elements within a range, use `[ start index : End Index ]`
- print complete list, use `[ : ]`
- For reverse list, use `[ : -1 ]`

Ex:

```
import array as arr
a = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
b = arr.array('i', a)
res = b[3:8]
print(res)
res = b[5:]
print(res)
res = b[: ]
print(res)
```

o/p:

```
array('i', [4, 5, 6, 7, 8])
array('i', [6, 7, 8, 9, 10])
array('i', [1, 2, 3, 4, 5, 6, 7, 8, 9, 10])
```

Searching Element

- In order to search an element in the array we use `index()` method.
- This function returns the index of the first occurrence of the value mentioned in arguments.

Ex: import array

```
a = array.array('i', [1, 2, 3, 1, 2, 5])
```

```
print(a.index(2))
```

```
print(a.index(1))
```

o/p:

1

0

Updating Elements

- In order to update an element in the array we simply reassigned a new value to the desired index we want to update.

Ex: import array

```
a = array.array('i', [1, 2, 3, 1, 2, 5])
```

```
a[2] = 6
```

```
print(a)
```

```
a[4] = 8
```

```
print(a)
```

o/p:

```
array('i', [1, 2, 6, 1, 2, 5])
```

```
array('i', [1, 2, 6, 1, 8, 5])
```


Operations on Arrays :

1. Counting Elements : we can use `count()` method to count given item in array.

ex: import array

O/P : 3

```
a = array.array('i', [1, 2, 3, 4, 2, 5, 2])
```

```
count = a.count(2)
```

```
print(count)
```

2. Reversing Elements : in order to reverse elements of an array use reverse method.

ex: import array

O/P : 5 4 3 2 1

```
a = array.array('i', [1, 2, 3, 4, 5])
```

```
a.reverse()
```

```
print(*a)
```

3. Extend Element : `extend()` function is used to attach an item for iterable to the end of the array. This method is used to add an array of values to the end of a given array.

ex: import array as arr

O/P :

```
a = arr.array('i', [1, 2, 3, 4, 5])
```

```
array('i', [1, 2, 3, 4, 5, 6,  
7, 8, 9, 10])
```

```
a.extend([6, 7, 8, 9, 10])
```

```
print(a)
```

Advanced Python

- object oriented programming empowers developers to build modular, maintainable and scalable applications.
- oop is a way of organizing code that uses objects and classes to represent real-world entities and their behaviour.
- In oop object has attributes thing that has specific data and can perform certain actions using methods.
- organizes code into classes and objects.
- supports encapsulation to group data and methods together.
- Enables inheritance for reusability and hierarchy.
- Allows polymorphism for flexible method implementation.

Class :

- A class is a collection of objects.
- classes are blueprints for creating objects.
- A class defines a set of attributes and methods that the created objects can have.
- classes are created by keyword `class`.
- Attributes are the variables that belong to a class.

• Attributes are always public and can be accessed using the dot (.).

operator. Example:

Myclass.Myattribute

creating a class

• Here, class keyword indicates that we are creating a class followed by name of the class.

```
ex: class Dog:
    species = "canine"
    def __init__(self, name, age):
        self.name = name;
        self.age = age
```

Explanation:

- class Dog: creates a class named Dog, which acts as a blueprint for dog objects.
- species is a class attribute, meaning it is shared by all instances of the class.
- __init__() is a constructor method that runs automatically when a new object is created. It is used to initialize object data.
- self refers to the current object, allowing each object to store and access its own data.
- self.name and self.age are instance attributes, unique to each dog object created from the class.

Objects:

- An object is an instance of a class.
- It represents a specific implementation of the class and holds its own data. An object consists of:

→ State: represented by the attributes and reflects the properties of an object.

→ Behaviour: represented by the methods of an object and reflects the response of an object to the other objects.

→ Identity: gives a unique name to an object and enables one object to interact with other objects.

creating object

• creating an object involves instantiating a class to create a new instance of that class. This process is also referred to as object instantiation.

Ex: class Dog:

species = "canine"

O/P: Buddy

def __init__(self, name, age):

Canine

self.name = name

self.age = age

dog1 = Dog("Buddy", 3)

print(dog1.name)

print(dog1.species)

Four pillars of OOP:

• four pillars of object oriented programming form the foundation for designing structured, reusable and maintainable software.

• The four pillars are:

1. Inheritance

2. Polymorphism

3. Encapsulation

4. Data Abstraction

1. Inheritance:

• Inheritance is a fundamental concept in object-oriented programming that allows a class to inherit attributes and methods from another class.

• Here, a parent class Animal is created that has a method info(). Then a child class Dog is created that inherits from Animal and adds additional behaviour.

Ex: class Animal:

def __init__(self, name):

self.name = name

def info(self):

print("Animal name:", self.name)

class Dog(Animal):

def sound(self):

print(self.name, "barks")

d = Dog("Buddy")

O/P: Animal name: Buddy

d.info()

Buddy barks

d.sound()

Benefits of Inheritance :

- promotes code reusability by sharing attributes and methods across classes.
- Models real-world hierarchies like Animal $\rightarrow$ Dog or Person $\rightarrow$ Employee
- Simplifies maintenance through centralized updates in Parent classes.
- Enables method overriding for customized subclass behavior.
- Supports scalable, extensible design using polymorphism.

super() function

- `super()` function is used to call methods from a superclass following Python's Method Resolution Order (MRO).
 - In particular, it is commonly used in the child class's `__init__()` method to initialize inherited attributes.
 - Thus way, the child class can leverage the functionality of the parent class.
- Ex : Here, Dog uses `super()` to call Animal's constructor.

class Animal:

```
def __init__(self, name):  
    self.name = name
```

```
def info(self):
```

```
    print("Animal name:", self.name)
```

```
class Dog(Animal):
```

```
    def __init__(self, name, breed):
```

```
        super().__init__(name)
```

```
        self.breed = breed
```

```
    def details(self):
```

```
        print(self.name, "is a", self.breed)
```

```
d = Dog("Buddy", "Golden Retriever")
```

```
d.info()
```

```
d.details()
```

o/p : Animal name: Buddy

Buddy is a Golden Retriever

Method overriding in Inheritance :

- Method overriding allows a child class to provide its own implementation of a ~~class~~ method that already exists in a parent class.
- This enables customized behaviour while still maintaining the inheritance relationship.

ex ÷ class Animal: o/p ÷ Bark
 def sound(self):
 print("Animal sound")
class Dog(Animal):
 def sound(self):
 print("Bark")
d = Dog()
d.sound()

Types of inheritance in python :

- Types of inheritance depend upon the no. of child and parent classes. There are four types of inheritance in python:

1. Single Inheritance

- Single Inheritance enables a derived class to inherit properties from a single parent class, thus enabling code reusability and the addition of new features to existing code

ex ÷ class parent:

```
def func1(self):  
    print("This function is  
in a parent class.")  
class Child(Parent):  
    def func2(self):  
        print("This function is  
in child class.")
```

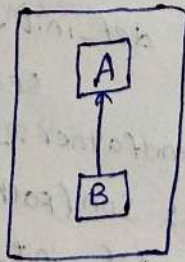
obj = Child()

obj.func1()

obj.func2()

o/p ÷ This function is in parent class.

This function is in child class.



2. Multiple Inheritance

- When a class can be derived from more than one base class this type of inheritance is called multiple inheritance.

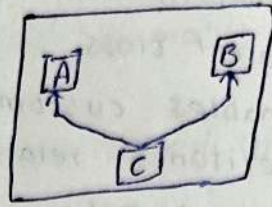
- In multiple inheritances, all the features of the base classes are inherited into the derived class.

Ex: class Mother:

```

    mothername = ""
    def mother(self):
    Print (self.mothername)
class Father:

```



```

    fathername = ""
    def father(self):
    Print (self.fathername)
class Son (Mother, Father):
    def parents(self):
        Print ("Father :", self.fathername)
        Print ("Mother :", self.mothername)

```

Sl = Son()

Sl.fathername = "RAM"

Sl.mothername = "SITA"

Sl.parents()

O/P:

Father : RAM

Mother : SITA

3. Multilevel Inheritance

- In multilevel inheritance, features of the base class and the derived class are further inherited into the new derived class. This is similar to relationship representing a child and a grandfather.

Ex: class Grandfather:

```

    def _init_(self, grandfathername):
        self.grandfathername = grandfathername
class Father(Grandfather):
    def _init_(self, fathername, grandfathername):
        self.fathername = fathername
Grandfather._init_(self, grandfathername)
class Son(Father):
    def _init_(self, sonname, fathername, grandfathername):
        self.sonname = sonname
Father._init_(self, fathername, grandfathername)
def print_name(self):
    Print ('Grandfathername:', self.grandfathername)
    Print ('fathername:', self.fathername)
    Print ('Son name:', self.sonname)

```

class Father(Grandfather):

def _init_(self, fathername, grandfathername):

self.fathername = fathername

Grandfather._init_(self, grandfathername)

class Son(Father):

def _init_(self, sonname, fathername, grandfathername):

self.sonname = sonname

Father._init_(self, fathername, grandfathername)

def print_name(self):

Print ('Grandfathername:', self.grandfathername)

Print ('fathername:', self.fathername)

Print ('Son name:', self.sonname)


```
print (S1.grandfathername)
```

```
S1.print_name()
```

o/p: Lal mani

Grandfather name: Lal mani

Father name: Rampal

Son name: prince



4. Hierarchical Inheritance

• More than one derived class are created from a single base this type of inheritance is called hierarchical inheritance.

• In this program, we have a parent class and two child classes.

Ex: class parent:

```
def func1(self):
```

```
    print("This function is in parent class.")
```

```
class child1(parent):
```

```
    def func2(self):
```

```
        print("This function is in child 1.")
```

```
class child2(parent):
```

```
    def func3(self):
```

```
        print("This function is in child 2.")
```

```
object1 = child1()
```

```
object2 = child2()
```

```
object1.func1()
```

```
object1.func2()
```

```
object2.func1()
```

```
object2.func3()
```

o/p: This function is in parent class.

This function is in child 1.

This function is in parent class.

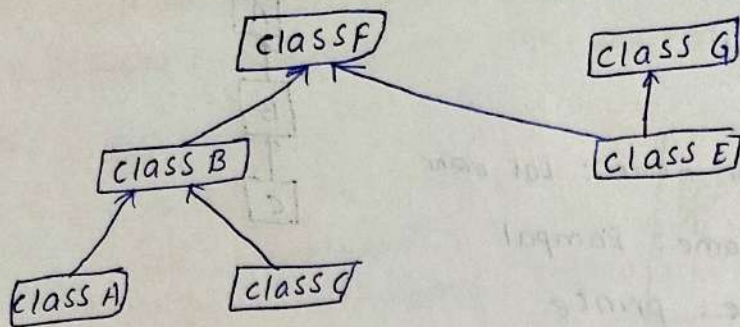
This function is in child 2.

5. Hybrid Inheritance

• Hybrid inheritance is a combination of more than one type of inheritance.

• It uses a mix like single, multiple or multilevel inheritance within the same program.

• Python's MRO handles such cases.



Ex: class school:

```

def func1(self):
    print("This function is in school.")

class Student1(school):
    def func2(self):
        print("This function is in student 1.")

class Student2(school):
    def func3(self):
        print("This function is in student 2.")

class Student3(Student1, school):
    def func4(self):
        print("This function is in student 3.")

obj = Student3()
obj.func1()
obj.func2()
  
```

O/P: This function is in school.

This function is in student 1.

2. Polymorphism

• polymorphism means "many forms" and allows the same method, function or operator to behave differently depending on the object or data it works with.

• This flexibility helps create more reusable, maintainable and scalable code.

• Real-life example: In a backend payment system, multiple payment options are available such as Debit card, credit card, UPI, Netbanking and wallet. All payment types use a common method named `processPayment()` but different implementations:

• Credit card payment

• UPI payment

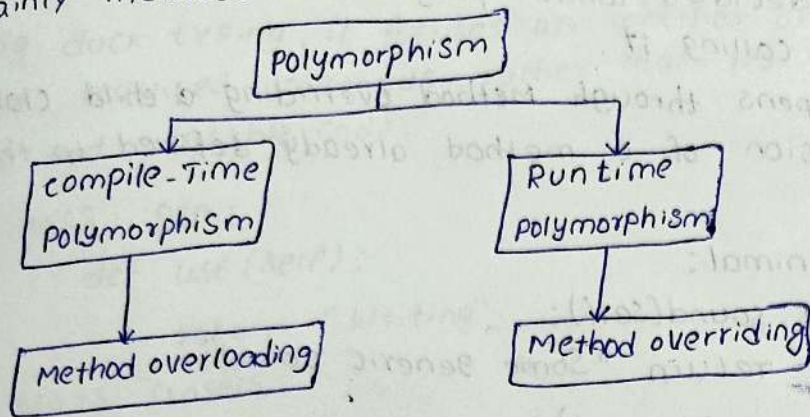
• wallet payment

• NetBanking payment

• The method name remains the same, but the action changes based on the payment type.

Types of polymorphism:

- polymorphism refers to the ability of the same method or operation to behave differently based on object or context.
- It mainly includes a compile-time and run-time polymorphism.



1. Compile-Time polymorphism

- compile-time polymorphism involves selecting a method or operation before program execution, typically through method or operator overloading.
- Language such as Java and C++ support this feature.
- Python does not support true compile-time polymorphism bcz method calls are resolved at runtime.
- However, similar behaviour can be achieved using default arguments, variable length arguments (*args) or keyword arguments (**kwargs)

Ex: class Calculator:

```
def multiply(self, a=1, b=1, *args):
```

```
    result = a * b
```

```
    for num in args:
```

```
        result *= num
```

```
    return result
```

```
calc = Calculator()
```

```
print(calc.multiply(1))
```

```
print(calc.multiply(4))
```

```
print(calc.multiply(2,3))
```

```
print(calc.multiply(2, 3, 4))
```

O/P:

1

4

6

24

- This code demonstrates overloading-like behaviour using default and variable-length arguments. While Python does not support the true method overloading, the same method can

handle different no. of arguments.

2. Runtime polymorphism

• Runtime polymorphism means that the behaviour of a method is decided while program is running, based on the object calling it.

• This happens through method overriding a child class provides its own version of a method already defined in the parent class.

Ex: class Animal:

```
def sound(self):
```

```
    return "Some generic sound"
```

```
class Dog(Animal):
```

```
def sound(self):
```

```
    return "Bark"
```

```
class Cat(Animal):
```

```
def sound(self):
```

```
    return "Meow"
```

```
animals = [Dog(), Cat(), Animal()]
```

```
for animal in animals:
```

```
    print(animal.sound())
```

o/p: Bark

Meow

Some generic sound

Polymorphism in Built-in Functions

• Python's built-in functions such as `len()` and `max()` are polymorphic because they work with different data types and return results based on type of object passed.

• This showcases its dynamic nature, where same function name adapts its behaviour depending on input.

```
Ex: print(len("Hello")) // String length
```

```
print(len([1, 2, 3])) // List length
```

```
print(max(1, 3, 2)) // Maximum of integers
```

```
print(max("a", "z", "m")) // maximum in string
```

o/p: 5

3

3

z

polymorphism in functions

- polymorphism allows functions to work with different object types as long as they support the required behavior.
- using duck typing, it focuses on whether an object has the required methods rather than its type, enabling flexible and reusable code.

```
ex ÷ class pen:
    def use(self):
        return "Writing"

class Erasers:
    def use(self):
        return "Erasing"

def perform_task(pen())
def perform_task(Eraser())
```

o/p ÷ Writing

Erasing

- Duck typing is a concept in python where the type of doesn't matter. what matter is whether it has the required methods or attributes.
- Here, ~~perform~~_task() doesn't check whether the object is a cat or Dog.
- It only assumes that the object has a sound() method. if it does, the function works.

Polymorphism in Operators

- Same operator can perform different tasks depending on operand types.
- This is known as operator overloading.
- This flexibility is a key aspect of polymorphism.

```
ex ÷ print(5+10)
```

// Integer addition

```
print("Hello " + "World!")
```

// String concatenation

```
print([1, 2] + [3, 4])
```

// List concatenation

o/p ÷ 15

Hello World!

[1, 2, 3, 4]

Method Overriding ÷

• Method overriding is an ability of any object-oriented programming language that allows a subclass or child class to provide a specific implementation of a method that is already provided by one of its super-classes or parent classes.

• When a method in subclass has the same name, the same parameters or signature, and same return type as a method in the subclass is said to override the method in the super-class.

Ex ÷

• `Super().__init__()` ÷ This ensures that the parent class's constructor is called, initializing any attributes defined in the parent class. It's good to practice to call the parent class constructor if it does important initialization.

• Method override ÷ The child class overrides the `show()` is called on an instance of child, it uses the child class's implementation.

Ex ÷

```
class parent():
```

```
    def __init__(self):
```

```
        self.value = "Inside parent"
```

```
    def show(self):
```

```
        print(self.value)
```

```
class Child(parent):
```

```
    def __init__(self):
```

```
        super().__init__()
```

```
        self.value = "Inside child"
```

```
    def show(self):
```

```
        print(self.value)
```

```
obj1 = parent()
```

```
obj2 = Child()
```

```
obj1.show()
```

```
obj2.show()
```

o/p ÷ Inside parent

Inside child

using super()

- Python `super()` function provides us the facility to refer the parent class explicitly.
- It is basically useful where we have to call superclass functions.
- It returns the proxy object that allows us to refer Parent class by 'super'.

Ex: `class parent():`

`def show(self):`

`print("Inside parent")`

`class child(parent):`

`def show(self):`

`super().show()`

`print("Inside child")`

`obj = child()`

`obj.show()`

o/p: Inside Parent

Inside child

Ex: `class Demo1:`

`def __init__(self):`

`print('HEY !!! Demo I am initialised (class Demo1)')`

`def sub-Demo(self, b):`

`print('Printing from class Demo1:', b)`

`class Demo2(Demo1):`

`def __init__(self):`

`print('HEY !!! Demo I am initialised (Demo2)')`

`super().__init__()`

`def sub-Demo(self, b):`

`print('Printing from class Demo2:', b)`

`super().sub-Demo(b+b)`

`class Demo3(Demo2):`

`def __init__(self):`

`print('HEY !!! Demo I am initialised (`

class Demo3):

super(). _init_(

def sub-Demo(self, b):

Print ('printing from class Demo3:', b)

super(). sub-Demo(b+1)

if __name__ == '__main__':

Demo = Demo3()

Demo.sub-Demo(10)

O/P ÷ HEY !!! Demo I am Initialised (class Demo3)

HEY !!! Demo I am Initialised (class Demo2)

HEY !!! Demo I am Initialised (class Demo1)

Printing from class Demo3: 10

Printing from class Demo2: 11

Printing from class Demo1: 12

Operator Overloading ÷

- operator overloading allows the same operator to perform different actions depending on the object it works with.

- Python supports operator overloading for both built-in data types and user-defined classes.

- operators such as + and * behave differently for different data types (for ex, numbers, strings and lists).

- user-defined classes can customize operator behaviour by implementing special methods such as _add_(), _sub_(),

- _mul_(), _eq_(), _lt_() and _gt_().

- The behaviour of built-in types cannot be changed globally, but custom classes can define their own operator implementations.

ex ÷ Print(1 + 2)

print ("Demo" + "For")

Print(3 * 4)

print ("Demo" * 4)

O/P ÷ 3

DemoFor

12

DemoDemoDemoDemo

User Defined TYPES:

- By default, Python does not know how operators should behave for objects of a custom class.
- To define this behaviour, python provides special methods that are automatically invoked when operators are used.
- Some commonly used operator methods are:

`__add__(self, other) → +`

`__sub__(self, other) → -`

`__eq__(self, other) → =`

- For ex. when we write `obj1 + obj2`. Python internally executes:

`obj1.__add__(obj2)`

Ex1: class A:

`def __init__(self, value):`

`self.value = value.`

`def __add__(self, other):`

`return self.value + other.value`

`obj1 = A(1)`

`obj2 = A(2)`

`print(obj1 + obj2)`

`print(A.__add__(obj1, obj2))`

`print(obj1.__add__(obj2))`

O/P ÷

3

3

3

Ex2: class Complex:

`def __init__(self, real, imag):`

`self.real = real`

`self.imag = imag`

`def __add__(self, other):`

`return self.real + other.real, self.imag + other.imag`

`c1 = Complex(1, 2)`

`c2 = Complex(2, 3)`

`print(c1 + c2)`

O/P ÷ (3, 5)

overloading comparison operators:

• operators like $>$, $<$ and $==$ can also be overloaded.

Ex-1 ÷ class A:

```
def __init__(self, a):
```

```
    self.a = a
```

```
def __gt__(self, other):
```

```
    return self.a > other.a
```

→ This code shows how to overload $>$ operator to compare two objects based on their stored values.

```
ob1 = A(2)
```

```
ob2 = A(3)
```

```
if ob1 > ob2:
```

```
    Print("ob1 is greater than ob2")
```

```
else:
```

```
    Print("ob2 is greater than ob1")
```

o/p ÷ ob2 is greater than ob1

Ex-2 ÷ class A:

```
def __init__(self, a):
```

```
    self.a = a
```

```
def __lt__(self, other):
```

```
    return "ob1 is less than ob2" if self.a < other.a else "ob2 is less than ob1"
```

```
def __eq__(self, other):
```

```
    return "Both are equal" if self.a == other.a else "Not equal"
```

```
ob1 = A(2)
```

```
ob2 = A(3)
```

```
print(ob1 < ob2)
```

```
ob3 = A(4)
```

```
ob4 = A(4)
```

```
Print(ob3 == ob4)
```

o/p ÷ ob1 is less than ob2
Both are equal

Non-Associative operators:

• Not all operators can be chained.

• Some are non-associative.

• For ex, $=$ and $+=$ cannot be combined in one statement.

Ex ÷ a = 5

b = 10

c = 15

a = b = (a < b) += (b < c)

o/p ÷ No output, bcz cannot be mixed in same expression, bcz

they are non-associative.

special Methods for Operator overloading :

python provides specific methods for each operator.

Binary operators

operator

Magic method

+

__add__(self, other)

-

__sub__(self, other)

*

__mul__(self, other)

/

__truediv__(self, other)

//

__floordiv__(self, other)

%

__mod__(self, other)

**

__pow__(self, other)

comparison operators

operator

Magic method

<

__lt__(self, other)

>

__gt__(self, other)

<=

__le__(self, other)

>=

__ge__(self, other)

==

__eq__(self, other)

!=

__ne__(self, other)

Assignment operators

operator

Magic method

=

__isub__(self, other)

+=

__isadd__(self, other)

*=

__imul__(self, other)

/=

__itruediv__(self, other)

//=

__ifloordiv__(self, other)

%=

__imod__(self, other)

**=

__ipow__(self, other)

Unary operators :

operator Magic Method

- `__neg__(self)`

+

`__pos__(self)`

~ `__invert__(self)`

Advantages :

- Improved readability - code looks natural (e.g., `obl + ob2` instead of `obl.add(ob2)`)
- consistency with built-in types - custom objects behave like integers, strings etc.
- Conciseness - less boilerplate code.
- custom-behaviour - extend operators to user-defined classes (e.g., vectors, matrices).
- Enhanced functionality - enable operators to work meaningfully with new data types.

3. Encapsulation

- Encapsulation is one of the concept of OOP.
- The idea of encapsulation is to bind the data members and methods into a single unit.
- Helps in better maintainability, readability and usability as we do not need to explicitly pass data members to member methods.
- Helps maintain data integrity by allowing validation and control over the values assigned to variables.
- It helps in achieving abstraction.
- A class can hide the implementation part and discloses only the functionalities required by other classes, which allows later changes to representations or implementations without impacting the codes that uses this class.

Ex ÷ class Employee:

```
def __init__(self, name, salary):
```

```
    self.name = name    // public attribute
```

```
    self.__salary = salary // private attribute
```

```
emp = Employee("Pedrick", 5000)
```

```
Print(emp.name)
```

```
Print(emp.__salary) // error
```

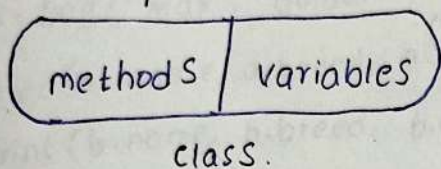
O/P ÷ Pedrick

Error! 'Employee' object has no attribute '__salary'.

explanation:-

- `self.name = name` : Public attribute, can be accessed directly.
- `self.salary = salary` : Private attribute, cannot be directly accessed.
- `print(emp.name)` : Prints "Fedrick" bcz name is public.
- `print(emp.__salary)` : Raises an error bcz `__salary` is private and hidden.

Encapsulation in python

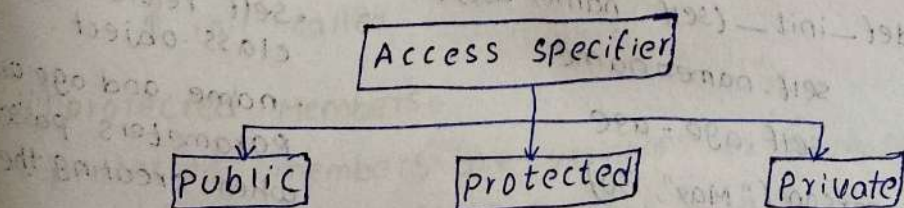


Why do we need Encapsulation?

- protects data from unauthorized access and accidental modification.
- controls data updates using getter/setter methods with validation.
- Enhances modularity by hiding internal implementation details.
- Simplifies maintenance through centralized data handling logic.
- Reflects real-world scenarios like restricting direct access to a bank account balance.

Access specifiers ÷

- Access specifiers define how class members (variables and methods) can be accessed from outside the class.
- They help in implementing encapsulation by controlling the visibility of data.
- There are three types of access specifiers:



i) public Members ÷

- public members are variables or methods that can be accessed from anywhere inside the class, outside

- the class or from other modules.
- By default all members in python are public.
- They are defined without any underscore prefix (e.g., self.name)

Ex ÷ class Employee:

```
def __init__(self, name):
    self.name = name // declared without _ . so it is public
def display_name(self):
    print(self.name)

emp = Employee("John")
emp.display_name()
print(emp.name)
```

O/P ÷ John

John

- display_name() ÷ public method that prints the value of public attribute.
- emp.name ÷ directly accessed from outside the class, showing public members are fully accessible.

__init__()

- It is a method
- __init__() method is a constructor which is automatically called when a new object of a class is created, its main purpose is to initialize the object's attributes and set up its initial state.
- when an object is created, memory is allocated for it and __init__ helps organize that memory by assigning values to attributes.

Ex ÷ class 'person':

```
def __init__(self, name, age):
    self.name = name
    self.age = age

a = person("Max", 30)
b = person("Emilia", 25)
```

O/P ÷ Max 30

Emilia 25

- self refers to current class object.
- name and age are parameters passed when creating the object.
- self.name and self.age are object attributes that store these values.

default parameters in `__init__()`

• Like regular functions, `__init__()` can have parameters with default values.

ex ÷ class Dog:

```
def __init__(self, name, breed="Mixed", age=1):
```

```
    self.name = name
```

```
    self.breed = breed
```

```
    self.age = age
```

```
a = Dog("Buddy")
```

```
b = Dog("Max", "Golden Retriever", 5)
```

```
print(a.name, a.breed, a.age)
```

```
print(b.name, b.breed, b.age)
```

o/p ÷ Buddy Mixed 1

Max Golden Retriever 5

• `breed="Mixed"` and `age=1` are default values.

• when creating `Dog("Buddy")` python uses the default values.

`__init__()` Method with Inheritance

• when using inheritance, both parent and child classes can have `__init__()` methods.

ex ÷ class A:

```
def __init__(self):
```

```
    print("A init called")
```

```
class B(A):
```

```
    def __init__(self):
```

```
        super().__init__()
```

```
        print("B init called")
```

```
obj = B()
```

o/p ÷ A init called

B init called

ii) protected members ÷

• protected members are variables or methods that are inherited to be accessed only within the class and its subclasses.

• They are not strictly private, but should be treated as internal.

- In python, protected members are defined with a single underscore prefix. (e.g., self._name)

Ex: class Employee:

```
def __init__(self, name, age):
    self.name = name
    self.age = age
class SubEmployee(Employee):
    def show_age(self):
        print("Age:", self._age)
emp = SubEmployee("Ross", 30)
print(emp.name)
emp.show_age()
```

O/P: Ross

Age: 30

iii) private members

- private members are variable or methods that cannot be accessed directly from outside the class.
- They are used to restrict access and protect internal data.
- In python, private members are defined with double underscore prefix. (e.g., self.__salary).
- Python uses name mangling, where the interpreter internally renames the variable (for eg, __salary becomes _ClassName__salary).
- This discourages direct access from outside the class, although it does not create strict privacy like other languages.

Ex: class Employee:

```
def __init__(self, name, salary):
```

```
    self.name = name
```

```
    self.__salary = salary
```

```
    def show_salary(self):
```

```
        print("Salary:", self.__salary)
```

```
emp = Employee("Robert", 60000)
```

```
print(emp.name)
```

```
emp.show_salary()
```


o/p: Robert

Salary: 60000

• Self. — Salary: Defined with double underscores. So it is private

Name mangling

• In python, name mangling prevents accidental name conflicts in classes, mainly during inheritance.

• It applies to variables or methods starting with two leading underscore (__) and not ending with double underscore.

-res.

• Python automatically renames such identifiers internally to avoid clashes, and this transformation is done by the interpreter, not the programmer.

• The name is transformed into the following format:

__classname__identifier

Ex: This ex shows how a variable with double underscores behaves inside and outside a class.

class Student:

def __init__(self, name):

self.__name = name

def show(self):

print(self.__name)

s = student("Jake")

s.show()

print(s.__name)

o/p:

Jake

ERROR!

'student' object has no attribute '__name'

Getter and Setter Methods

• In python, getter and setter methods are used to access and modify private attributes safely.

• Instead of accessing private data directly, these methods provide controlled access, allowing you to:

• Read data using a getter method.

• Update data using a setter method with optional validation or restriction.

Ex: class Employee:

```
def __init__(self):
```

```
    self.__salary = 50000
```

```
def get_salary(self):
```

```
    return self.__salary
```

```
def set_salary(self, amount):
```

```
    if amount > 0:
```

```
        self.__salary = amount
```

```
    else:
```

```
        print("Invalid salary amount!")
```

```
emp = Employee()
```

```
print(emp.get_salary())
```

```
emp.set_salary(60000)
```

```
print(emp.get_salary())
```

O/P : 50000
60000

• `__salary` is a private attribute.

• `get_salary()` is a getter method.

• `set_salary(amount)` is a setter method that updates the salary only if the amount is positive.

4. Data Abstraction

• Abstraction is the process of hiding implementation details and exposing only the essential functionality to the user.

• It is used to hide the implementation details from the user and expose only necessary parts, making the code simpler and easier to interact with.

• Real life Ex: In a graphics software, multiple shapes are available such as circle, Rectangle, and Triangle.

• All shapes share common data and support the same set of actions, but the internal details are hidden:

• Common Data: name, color, border, thickness, fill style.

• Common Actions: draw(), resize(), getArea().

• Each shape implements these actions differently circle uses radius, Rectangle uses length and width and

Triangle uses base and height.

Abstract Base Class:

- Abstract Base Class (ABC) is used to achieve abstraction by defining a common interface for its subclasses.
- It cannot be instantiated directly and serves as a blueprint for other classes.
- Abstract classes are created using abc module and @abstractmethod decorator, allowing developers to enforce method implementation in subclasses while hiding complex internal logic.

Ex: from abc import ABC, abstractmethod

```
class Greet(ABC):
```

```
    def say_hello(self):
```

```
        pass
```

```
class English(Greet):
```

```
    def say_hello(self):
```

```
        return "Hello!"
```

```
g = English()
```

```
print(g.say_hello())
```

o/p: Hello!

Components of Abstraction:

- Abstraction is made up of key components, these elements work together to define a clear and enforced structure for subclasses while hiding unnecessary implementation details.

1. Abstract Method

- Abstract methods are method declarations without a body defined inside an abstract class.
- They act as placeholders that force subclasses to provide their own specific implementation, ensuring consistent structure across derived classes.

Ex: from abc import ABC, abstractmethod

```
class Animal(ABC):
```

```
    def make_sound(self):
```

```
        pass
```

o/p: NO output.

make_sound() is an abstract method in Animal class, so it doesn't have any code inside it.

2. Concrete Method

- Concrete methods are fully implemented methods within an abstract class.
- Subclasses can inherit and use them directly, promoting code reuse without needing to redefine common functionality.

Ex: from abc import ABC, abstractmethod

```
class Animal(ABC):
```

```
    def make_sound(self):
```

```
        pass
```

```
    def move(self):
```

```
        return "Moving"
```

```
class Dog(Animal):
```

```
    def make_sound(self):
```

```
        return "Bark"
```

```
dog = Dog()
```

```
Print(dog.move())
```

o/p: Moving

3. Abstract properties

- Abstract properties work like abstract methods but are used for properties.
- These properties are declared with @property decorator and marked as abstract using @abstractmethod.
- Subclasses must implement these properties.

Ex: from abc import ABC, abstractmethod

```
class Animal(ABC):
```

```
    def species(self):
```

```
        pass
```

```
class Dog(Animal):
```

```
    def species(self):
```

```
        return "Canine"
```

```
dog = Dog()
```

```
Print(dog.species)
```

o/p: Canine

4. Abstract class instantiation

- Abstract classes cannot be instantiated directly.
- This is because they contain one or more abstract methods or properties that lack implementations.
- Attempting to instantiate an abstract class results in a **TypeError**.

Ex: from abc import ABC, abstractmethod

```
class Animal(ABC):
```

```
    def make_sound(self):  
        pass
```

```
animal = Animal()
```

- Instantiating `Animal()` raises a `TypeError` bcz abstract classes with unimplemented methods can't be instantiated, only fully implemented subclasses can.

Exception Handling

- Exception handling allows a program to handle unexpected errors during execution in a controlled way, instead of crashing abruptly.
- It enables programs to detect errors, manage them properly and continue execution whenever possible.
- Handles runtime errors such as invalid input, file not found, division by zero and type mismatches that occur during program execution.
- Helps improve program reliability by ensuring the applications does not terminate unexpectedly when an error occurs.

Below is a basic example demonstrating how to catch an exception and handle it gracefully.

Ex: `n = 10`

```
try:
```

```
    res = n / 0
```

```
except ZeroDivisionError:
```

```
    print("Can't be divided by zero!")
```

O/P: Can't be divided by zero!

- A `ZeroDivisionError` in python occurs when we try to divide a number by 0. we can't divide a number by 0 otherwise it will raise an error.

Syntax: python provides four main keywords for handling exceptions: `try`, `except`, `else` and `finally` each plays a unique role.

```
try:
```

```
    # code
```

```
except SomeException:
```

```
    # code
```

```
else:
```

```
    # code
```

```
finally:
```

```
    # code
```


- try: Runs the risky code that might cause an error.
- except: Catches and handles the error if one occurs.
- else: Executes only if no exception occurs in try.
- finally: Runs regardless of what happens useful for cleanup tasks like closing files.

Ex: try:

n = 0

res = 100 / n

except ZeroDivisionError:

Print("You can't divide by zero!")

except ValueError:

Print("Enter a valid number!")

else:

Print("Result is", res)

finally:

Print("Execution Complete.")

O/P: You can't divide by zero!

Execution complete.

Catching Exceptions:

- we can handle errors more efficiently by specifying the types of exceptions we expect. This can make code both safer and easier to debug.

1. Specific Exceptions

- Catching specific exceptions makes code to respond to different exception types differently.
- It precisely makes your code safer and easier to debug.
- It avoids masking bugs by only reacting to the exact problems you expect.

Ex: try:

x = int("str")

inv = 1/x

except ValueError:

Print("NOT valid!")

except ZeroDivisionError:

Print("zero has no inverse!")

O/P: NOT valid!

2. Multiple Exceptions

we can catch multiple exceptions in a single block if we need to handle them in the same way or we can separate them if different types of exceptions require different handling.

ex: `a = ["10", "twenty", 30]`

try:

`total = int(a[0]) + int(a[1])`

`except (ValueError, TypeError) as e:`

`print("Error", e)`

`except IndexError:`

`print("Index out of range.")`

o/p: Error invalid literal for int() with base '10: 'twenty'

3. Catch-All Handlers and their Risks

Catch-all handler is used to call to catch any exception (similar to else statement). use only except keyword to define it.

ex: try:

`res = "100" / 20`

`except ArithmeticError:`

`print("Arithmetic problem.")`

`except:`

`print("Something went wrong!")`

o/p: Something went wrong!

Raise an exception

we raise an exception using the raise keyword followed by an instance of the exception class that we want to trigger.

we can define our own custom exceptions by inheriting from python's built-in exception class.

Syntax: `raise ExceptionType("error message")`

raise: The raise keyword raises an exception and immediately stops the normal execution of the program. The control is then transferred to the nearest matching except block, otherwise the program terminates.

ex: `num = -3`

`if num < 0:`

`raise Exception("Negative numbers are not allowed")`

o/p: Exception: Negative numbers are not allowed.


```

Ex ÷ def set(age):
    if age < 0:
        raise ValueError("Age Cannot be negative")
    print(f"Age set to {age}")
try:
    set(-5)
except ValueError as e:
    print(e)

```

OIP ÷ Age Cannot be negative.

Errors vs Exceptions:

• Errors and exceptions are both issues in a program, but they differ in severity and handling.

Error: Issues in the program logic such as `SyntaxError`, etc. It occurs at compile time.

Exception: problems that occur at runtime and can be managed using exception handling. (e.g., invalid input, missing files).

File Handling in Python:

- File handling refers to the process of performing operations on a file, such as creating, opening, reading, writing and closing it through a programming interface.
- It involves managing the data flow between the Program and the file system on the storage device, ensuring that data is handled safely and efficiently.

Needs for File Handling

- Store data permanently, even after the program ends.
- Access external files like `.txt`, `.csv`, `.json` etc.
- Process large files efficiently without using much memory.
- Automate tasks like reading configs or saving outputs.

Opening a File

- To open a file, we can use `open()` function which requires file-path and mode as arguments.
- `open()` function is used to open a file in python.
- It allows you to read data from a file, write new data, append content or create a new file depending on the mode specified.

returns a file object that can be used to perform operations.

Syntax: `file = open('filename.txt', 'mode')` → It is for opening a file.

`filename.txt` → name or path of the file to be opened.

`mode` → mode in which you want to open the file.

• If you don't specify any mode, python uses 'r' (readmode) by default.

Ex: `f = open("data.txt", "r")` o/p: Hello World
`print(f.read())`
`f.close()`

Syntax for open()

`open(file, mode="r")`

Common file modes:

"r" — opens a file for reading.

"w" — opens a file for writing and overwrites existing content.

"a" — opens a file for appending content.

"x" — creates a new file and raises an error if it already exists.

"b" — opens a file in binary mode.

"t" — opens a file in text mode (default)

Ex: `f = open("message.txt", "w")`
`f.write("Python File Handling")`
`f.close()`
`f = open("message.txt", "r")`
`print(f.read())`
`f.close()`

o/p: Python File Handling

Ex: `f = open("log.txt", "a")`

`f.write("In New entry added")`

`f.close()`

`f = open("log.txt", "r")`

`print(f.read())`
`f.close()`

o/p: Application started
New entry added

Basic example for opening and closing File

```
f = open("geek.txt", "r")
```

```
print(f)
```

```
f = open("data.txt", "r")
```

```
f.close()
```

Checking file Properties

- Once the file is open, we can check some of its properties:

```
f = open("data.txt", "r")
```

```
print(filename:", f.name)
```

```
print("Mode:", f.mode)
```

```
print("Is closed?", f.closed)
```

```
f.close()
```

```
print("Is closed?", f.closed)
```

O/P: filename: data.txt

Mode: r

Is closed? false

Is closed? True

Reading a file

- Reading a file can be achieved by file.read() which reads the entire content of the file.
- After reading, it's good practice to close the file to free up system resources.

```
ex: f = open("data.txt", "r")
```

```
content = f.read()
```

```
print(content)
```

```
f.close()
```

O/P: Hello World

Data Definition

123 456

Writing a file

- Writing to a file is done using the mode "w".
- This creates a new file if it doesn't exist, or overwrites the existing file if it does.
- The write() method is used to add content.
- After writing, make sure to close the file.

```
ex: with open("data.txt", "w") as file:
```

```
file.write("Hello, Python!\n")
```

```
file.write("File handling is easy with Python.")
```

```
print("File written Successfully")
```

O/P: Hello, Python!

File handling is easy with python

using with statement

- Instead of writing and closing the file, you can use the with statement, which automatically handles closing. This reduces the risk of file corruption and resource leakage.
- Replaces long try-except-finally blocks with cleaner syntax.
- Improves readability by reducing unnecessary boilerplate code.

ex: with open("data.txt", "r") as file:

content = file.read()

print(content)

o/p: Hello, World!

Handling exceptions when closing a file

- It's imp to handle exceptions to ensure that files are closed properly, even if an error occurs during file operations.
- Here, the finally block ensures the file is closed even if an error occurs.

ex: try:

file = open('data.txt', 'r')

content = file.read()

print(content)

except FileNotFoundError as e:

print("Error:", e)

finally:

file.close()

o/p: Hello, World!

Packages

- Python packages are a way to organize and structure code by grouping related modules into directories.
- A package is essentially a folder that contains an `__init__.py` file and one or more python files (modules).
- Allows modules to be easily shared and distributed across different applications.

Key Components of a python package

- **Module**: A single python file containing reusable code. (Math.py)
- **Package**: A directory containing modules and special `--init--.py` file.
- **sub-Package**: packages nested within other packages for deeper organization.

key characteristics of packages:

- Packages are collections of modules that organize related code, making it easier to reuse and share.

1. Namespace Management
2. Reusability
3. Dependency Management.

Why use packages:

1. Organized code structure.
2. Easy code reuse
3. Simplifies sharing & Distribution (pip)
4. Makes large projects manageable.

Creating and Accessing Packages:

1. **Create a Directory**: create a folder that will act as Package root.
2. **Add Modules**: inside the directory, add python files (modules). Each module can contain related functions or classes.
3. **Add `--init--.py`**: Add an `--init--.py` file to the directory. This file tells python that the directory should be treated as a package.
4. **Create sub-Packages**: one can organize code further by creating subdirectories with their own `--init--.py` files.
5. **Import modules**: modules or functions inside the package can be imported using dot notation. for ex:

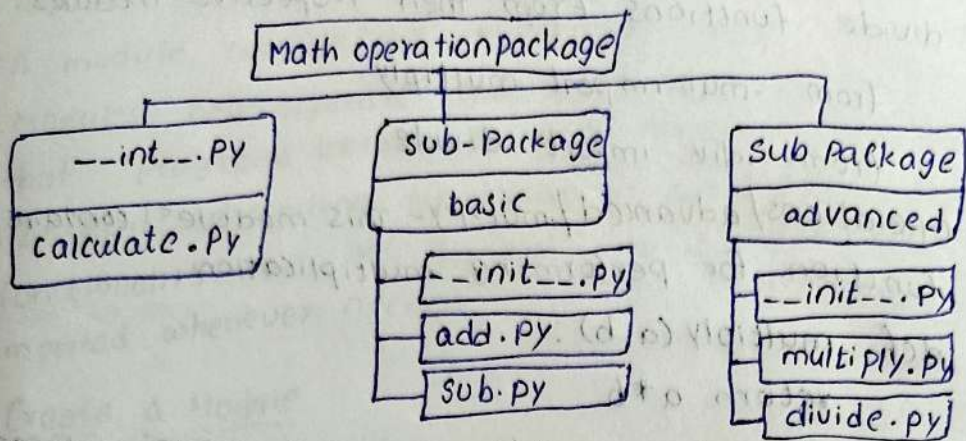
```
from myPackage.module1 import greet
```

Ex:

In this example, we create a Package called `math-operations` to organize mathematical functions. The package contains two sub-Packages:

- `basic` - Addition and subtraction
- `advanced` - multiplication and division

Each operation is stored in its own module, which makes the code modular, reusable and easier to maintain.



math-operations/--init---.py - This file initializes the main Package and exposes commonly used functions so they can be imported directly from the package.

```
from .calculate import calculate
from .basic import add, subtract
from .advanced import multiply, divide
```

math-operations/calculate.PY - This module contains a simple function that prints a message indicating that calculation is being performed.

```
def calculate():
    print("performing calculation")
```

math-operations/--init---.py - This file initializes the basic sub-package and makes the add and subtract functions available when the package is imported.

```
from .add import add
from .sub import subtract
```

math-operations/basic/add.PY - This module contains the functions for performing addition.

```
def add(a, b):
    return a+b
```

math-operations/basic/sub.PY - This module contains the functions for performing subtraction.

```
def subtract(a, b):
    return a-b
```


math-operations/advanced/___init__.py - this file initializes the advanced sub package and exposes the multiply and divide functions from their respective modules.

```
from .mul import multiply
```

```
from .div import divide
```

math-operations/advanced/mul.py - this module contains the function for performing multiplication.

```
def multiply(a,b):
```

```
    return a*b
```

math-operations/advanced/div.py - This module contains the function for performing division.

```
def divide(a,b):
```

```
    return a/b
```

Using the Package: Now imports functions from the package and use them in our program.

```
from math-operations import calculate, add, subtract,  
multiply, divide
```

```
calculate()
```

```
print("Addition:", add(5,3))
```

```
print("Subtraction:", Subtract(10,4))
```

```
print("Multiplication:", multiply(4,2))
```

```
print("Divison:", divide(10,2))
```

O/P:

Performing calculation...

Addition: 8

Subtraction: 6

multiplication: 8

Divison: 5.0

Modules

- module is a file containing definitions and statements.
- A module can define functions, classes and variables.
- Modules help organize code into separate files so that programs become easier to maintain and reuse.
- Instead of writing everything in one place, related functionality can be grouped into its own module and imported whenever needed.

Create a Module

- To create a module, write the desired code and save that in a file with .py extension.

```
ex: #calc.py
def add(x, y):
    return (x+y)

def subtract(x, y):
    return (x-y)
```

This is all that required to create a module.

Import Module

- Module can be used in another file using the import statement.
- When python sees an import, it loads the module if it exists in the interpreter's search path.

Syntax: import module

```
ex: import calc
print(calc.add(10, 2))
```

o/p: 12

Types of Import Statements:

1. Import From Module: This allows importing specific functions, classes or variables rather than the whole module.

```
from math import sqrt, factorial
print(sqrt(16))
```

- 4.0

```
print(factorial(6))
```

- 720

2. Import All Names : * imports everything from a module into the current namespace.

```
from math import *
```

```
Print(sqrt(16))
```

```
Print(factorial(6))
```

```
4.0
```

```
720
```

3. Import with Alias: You can shorten a modules name using as.

```
import math as m
```

```
Print(m.pi)
```

```
3.141592653589793
```

Types of Modules :

1. Built-in Modules: These come bundled with Python and require no installation.

eg - math, random, os

```
import random
```

```
Print(random.randint(1,5))
```

```
- 4
```

2. User-defined Modules: These are modules you create yourself, such as calc.py

```
import calc
```

```
print(calc.subtract(20,5))
```

```
- 15
```

3. External (Third-party) Modules: These modules are installed using PIP - eg., NumPy, Pandas, Requests.

```
import requests
```

```
r = requests.get("https://example.com")
```

```
print(r.status-code)
```

```
- 200
```

4. Package modules: A Package is a directory containing multiple modules, usually with an `--init--.py` file.

Example Directory

```
myPkg/  
  --init--.py  
  calc.py  
  utils.py
```

• Using a module from a Package

```
from myPkg import utils  
print(utils.some_func())
```

- calls a function named `some_func()`, the o/p will be whatever the function returns.

- if `utils.py` contains something like:

```
def some_func():  
    return "Hello"
```

o/p: Hello

Locating a Module

• python searches for modules in a predefined list of directories known as the module search path. you can view that list using `sys.path`.

```
import sys
```

```
for p in sys.path:
```

```
    print(p)
```

o/p: /home/guest/sandbox

/usr/local/lib/python3.2ip

/usr/local/lib/python3.13

/usr/local/lib/python3.13/lib-dynload

/usr/local/lib/python3.13/site-packages

- python checks each path in order until it finds the module you are trying to import.